

T34 vs Tiger

Editor Manual

Updated by Panzersim & Murkz November 2009, PZULArgetool December 2011

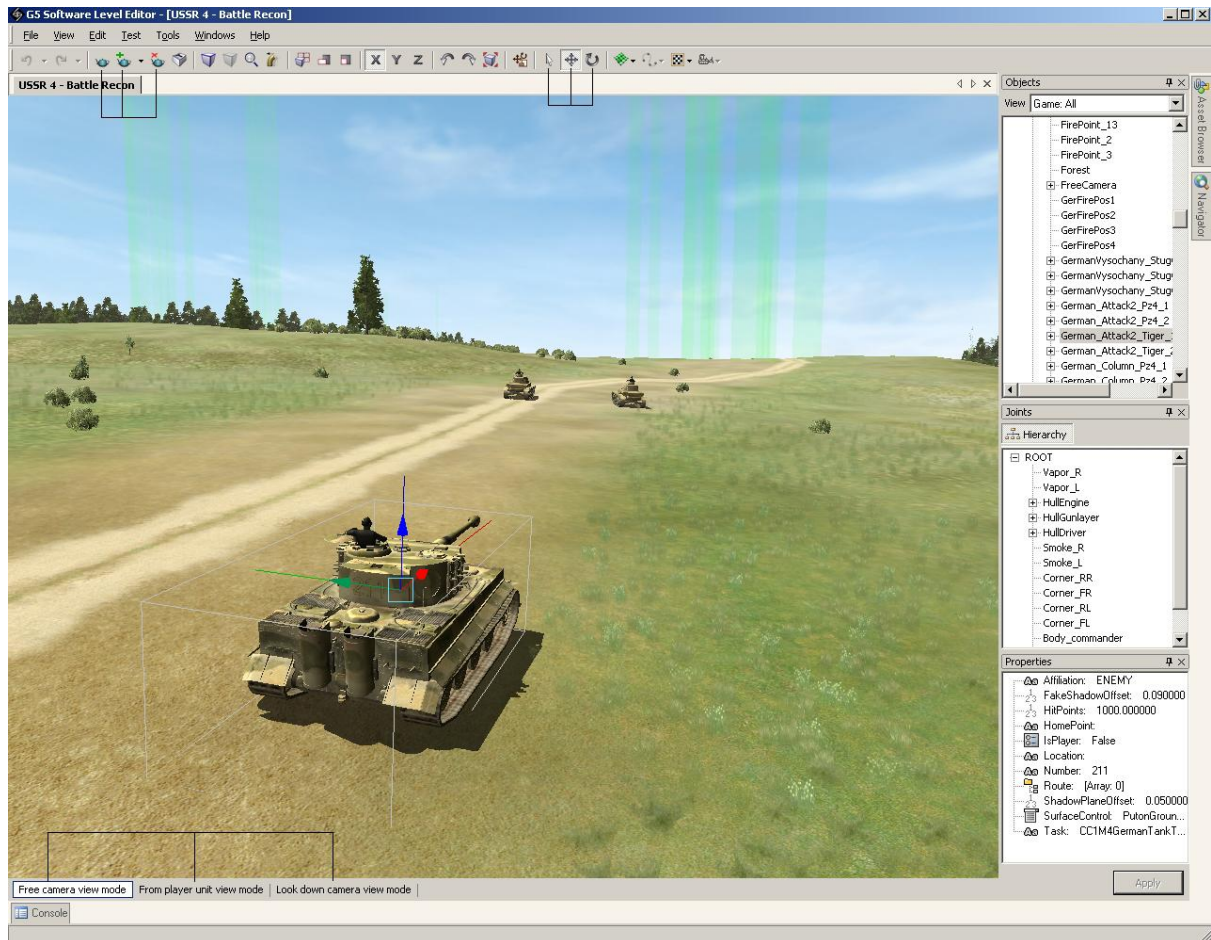
<http://panzersim.forumotion.com/forum.htm>



Contents

Camera control and navigating around the map.....	3
Adding objects units on the map	4
Properties of objects and units	7
Way-Points (NavPoint).....	8
The directory structure of the mission	10
Editing landscape.....	11
Navigator	13
Editing vegetation	14
Editing roads.....	16
Editing microtexture	17
Log and Console.....	19
Ivan's Scenario Trutorial	20
Lesson 1. Create your own mission.	20
Adding a new terrain and scenario from scratch.....	46

Camera control and navigating around the map



The camera in the editor has multiple viewing modes - free, player unit (see you're unit) and top view. Toggle the camera using the buttons in the lower left corner of the screen.

<Shift> (Move camera up)

<Ctrl> (Move camera down)

<Mouse wheel forwards>

<Mouse wheel backwards>

On the numeric keypad

<NumLock> (Change to camera mode, when enabled you can fly around the map),

<*> (Max speed cameras)

</> (Min. speed cameras)

The arrows are used to move and rotate the selected or added facilities

In addition, the free camera can be directed to any object in the mission by clicking on the object name in the list on the side-menu objects.

You can also move the camera with these duplicated buttons <W> <A> <S> <D>.

Adding objects units on the map

Objects are added using the buttons in the top menu of the editor, you manipulate objects the same way using the numpad (select, move, rotate). Selection of an object is done by clicking the RMB on the object, in the free camera mode (NumLock) or with the tab key.

Make sure the object is not blocked by any other objects — what can happen that the selection is not an object. After selecting an object, manipulation can be performed using the numeric keypad.

Clicking the middle mouse button on it secures center of the camera view to the selected object and allows you to move it along with the camera re-clicking on middle mouse button disables this mode.

Similarly, the object can be selected for manipulation in the menu objects. In addition, using the Insert key can clone the selected object.

Save edits made using the menu bar File> Save Level. The level will be saved in a file in the directory Content.script mission. (This file must be writable, it is desirable to remove read-only with all files being edited missions, so that you do not accidentally lose the results of the changes that are not in write-protected file).

The main difference between units from other objects (buildings, fences, etc.) is the need to control their behavior according to the scenario of the mission. Management is applied to class groups of units and their task behavior using tasks (Taxco).

See this example in the first mission of the Soviet campaign: One of the T34-76, the one starting left of the player is PlayerTank_1, and belongs to the group CC1M1Gr_PlayerTanks. The file Content.script mission for the unit has an entry:

```
[
  "PlayerTank_1",
  "GameObject",
  "CTankT34_76_42Unit",
  new Matrix(
    0.991726, 0.127028, -0.018517, 3688.695801,
    -0.126472, 0.991557, 0.028631, 3702.108887,
    0.021998, -0.026052, 0.999418, 573.912415,
    0.000000, 0.000000, 0.000000, 1.000000
  ),
  [
    ["BehRadarMask", [{"ENEMY", "MainMesh"}, {"FRIEND", "INVISIBLE_ON_RADAR"}]],
    ["Task", "CC1M1Tsk_USSR_TanksRadar"],
    ["Affiliation", "FRIEND"],
    ["Number", "22_13"]
  ]
],
```

Where ["Task", "CC1M1Tsk_USSR_TanksRadar"], is an indication of the individual TASK unit, in this case ("CC1M1Tsk_USSR_TanksRadar") from the MissionTasks.script. If it is not

necessary to prescribe any specific action the task unit, you can use the basic classes of tasks. For example, for tanks - CBaseAITankTask, for the ACS - CBaseAISAUTask, for the APC - CBaseAIBtrTask, in other cases, you can use CBaseAITask. To control units through the management class groups create a group, which may also consist of a single unit.

```
[ "CC1M1Gr_PlayerTanks",
  "UnitGroup",
  "CC1M1Gr_PlayerTanks",
  new Matrix (
    0.991688, 0.127003, -0.020595, 3688.695801,
    -0.126419, 0.991595, 0.027523, 3702.108887,
    0.023918, -0.024691, 0.999409, 574.211975,
    0.000000, 0.000000, 0.000000, 1.000000
  )
  [
    [ "Units", [ "PlayerTank_1", "PlayerTank_2"]],
    [ "Path", [ "NP_PlayerTanks_PP_1", "NP_PlayerTanks_PP_2", "NP_PlayerTanks_PP_3",
      "NP_PlayerTanks_PP_4"]],
    [ "DelayedOrder", true],
    [ "Formation", "Column"],
    [ "FormationDistance", 17],
    [ "MovingSpeed", 8.800000],
    [ "FirstOrder", "Patrol"],
    [ "CyclePath", false]
  ]
]
```

Where

"CC1M1Gr_PlayerTanks",	- name of the group
"UnitGroup",	- the type of group (units)
"CC1M1Gr_PlayerTanks",	- managing a group class in the file MissionTasks.script
new Matrix (	
0.991688, 0.127003, -0.020595, 3688.695801,	
-0.126419, 0.991595, 0.027523, 3702.108887,	
0.023918, -0.024691, 0.999409, 574.211975,	

0.000000, 0.000000, 0.000000, 1.000000

) - The matrix of, the group does not matter.

["Units", ["PlayerTank_1", "PlayerTank_2"]], - the group and type of components (units)

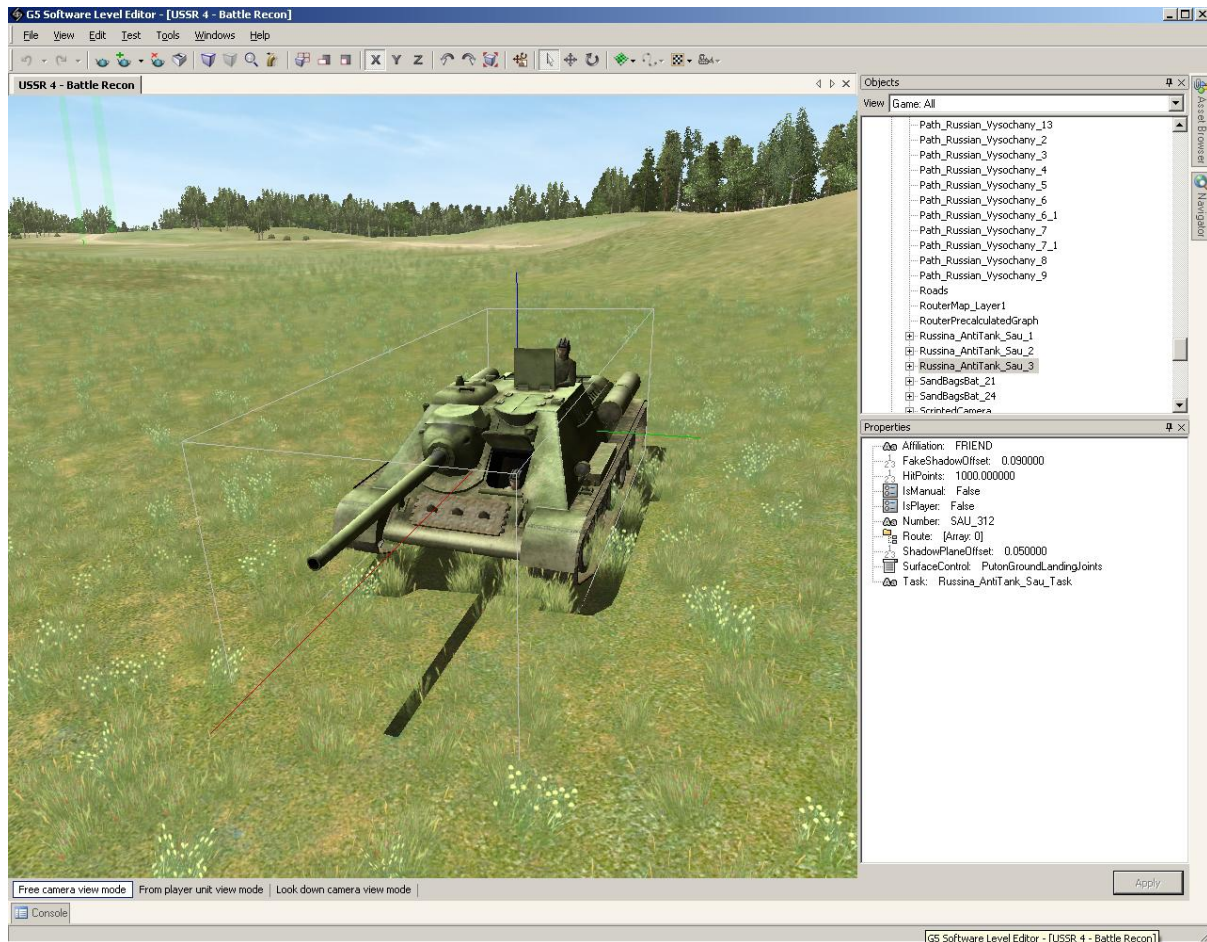
["Path", ["NP_PlayerTanks_PP_1", "NP_PlayerTanks_PP_2", "NP_PlayerTanks_PP_3",
"NP_PlayerTanks_PP_4"]],

- the way-points for the group.

The group's properties can be specified, including the reaction to enemy units, for example:

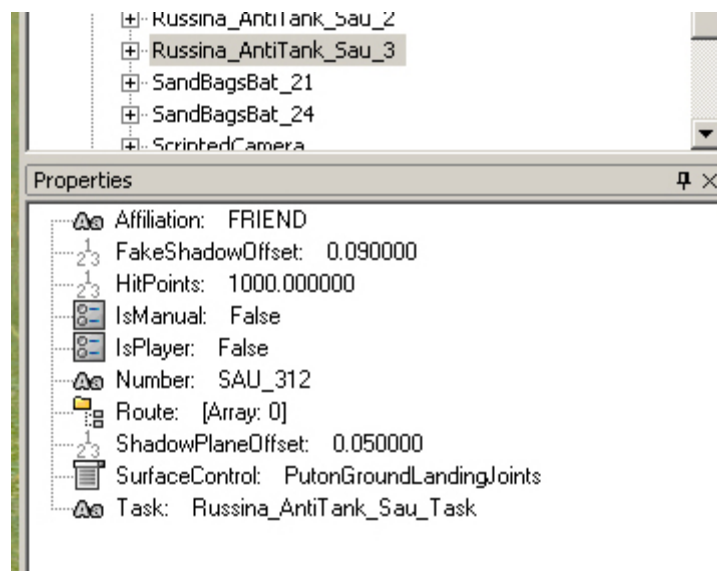
["GroupEnemyReaction", "ERT_AGGRESSIVE"], can use values ERT_AGGRESSIVE - aggressive behavior, ERT_PASSIVE - passive defensive behavior, ERT_FRIGID – completely ignoring the enemy.

Properties of objects and units



When you select an object in the Properties menu displays a list of properties of the selected object,

Here are some of the most important:



Affiliation	- can have the values of FRIEND, ENEMY, NEUTRAL. These indicate the unit belongs to either FRIEND - compatriot, ENEMY - enemy, NEUTRAL- neutral.
HitPoints	- hitpoints act on different objects than technology that is buildings, barrels, shafts, infantry, etc. Are mostly used for buildings, machines have no effect. IsManual and IsPlayer set to True only for the tank player, one on the map.
Number	- name of the texture unit numbers.
SurfaceControl	- a method for setting landshaft.
Task	- Managing a script for this unit. (Text of the script is in the file MissionTasks.script edited directory of the mission.)

Way-Points (NavPoint)

You specify the route used by a list of waypoints (NavPoint), indicating the group of units route their traffic (the text of the script is in the file in the directory edited Content.script, "PATH" section for the respective group.)

For example, one group of tanks from the first mission of the Soviet campaign

```

"CC1M1Gr_PlayerTanks",
"UnitGroup",
"CC1M1Gr_PlayerTanks",
new Matrix(
0.991688, 0.127003, -0.020595, 3688.695801,
-0.126419, 0.991595, 0.027523, 3702.108887,
0.023918, -0.024691, 0.999409, 574.211975,
0.000000, 0.000000, 0.000000, 1.000000
),
[
["Units", ["PlayerTank_1", "PlayerTank_2"]],
["Path", ["NP_PlayerTanks_PP_1", "NP_PlayerTanks_PP_2", "NP_PlayerTanks_PP_3",
"NP_PlayerTanks_PP_4"]],
["DelayedOrder", true],

```

```

["Formation", "Column"],
["FormationDistance", 17],
["MovingSpeed", 8.800000],
["FirstOrder", "Patrol"],
["CyclePath", false]
]
],

```

Units	- The group, in this case 2 tanks. ["Units", ["PlayerTank_1", "PlayerTank_2"]],
NP Path	-navpoint, which navigates the group.
DelayedOrder	- delayed execution of the order. (tanks will go on the route after a certain event, the event itself stated in the script of the mission, in a file mission.script, delayed the order to begin the movement there looks like:sendEvent (0.0, "CC1M1Gr_PlayerTanks", "PopDelayedOrder ",[])). Usually exposed to False.
Formation	- the formation of the group.
FormationDistance	- intervals between the center of the units in the formation.
MovingSpeed	- group speed.
CyclePath	- To carry on in a loop

To install a way point in the editor, you must first turn on the display for navpoints:

View> Special objects> Nav Points. The way-point will appear in the centre of you're point of view.

Then through the drop-down list button add items Create Object () choose

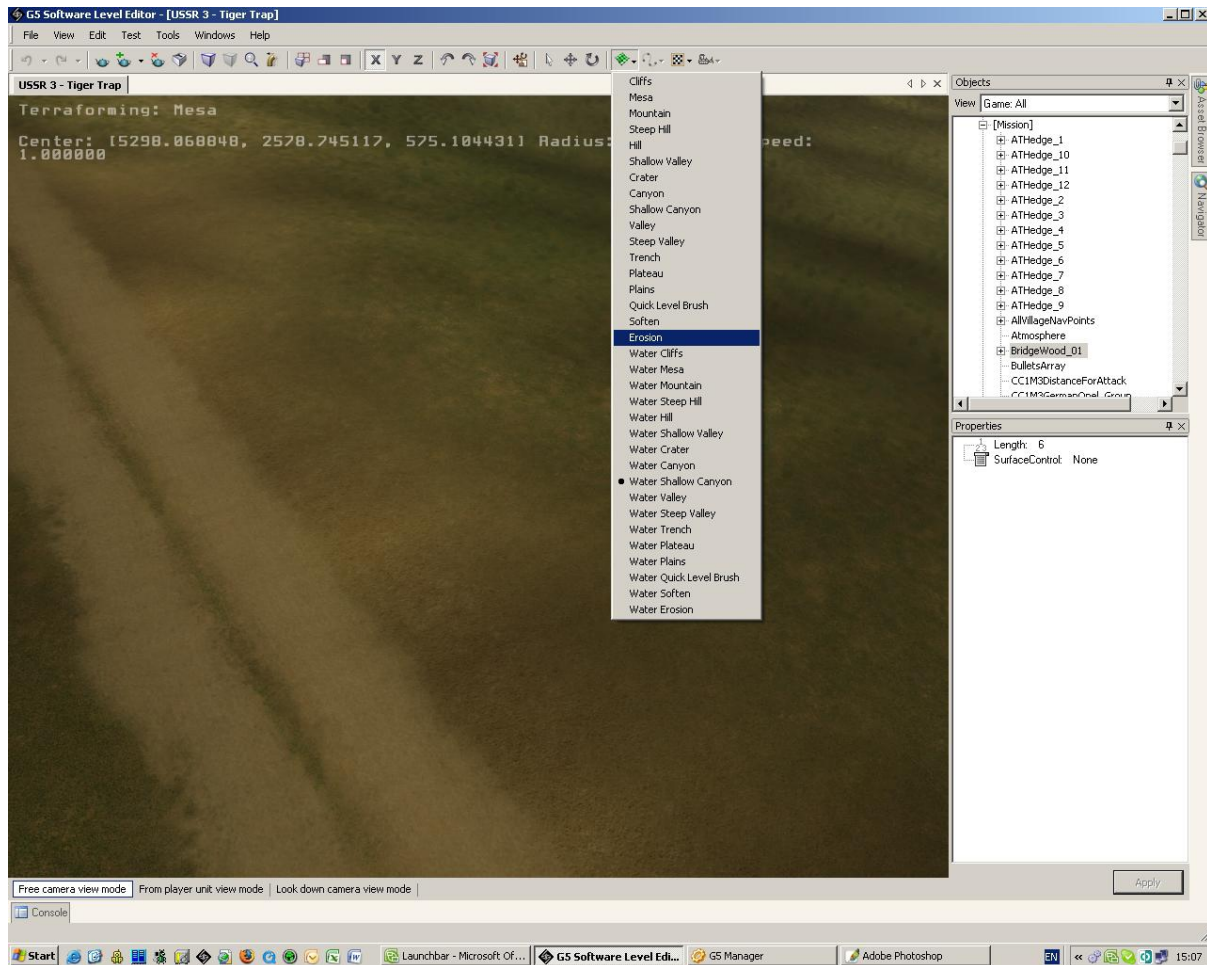
Special Objects> Navigation Points> Z Axis Cylinder and click Create Object (), in the center of the field view camera is created navpoint.

The directory structure of the mission

Mission folders contain the following files, which are used directly by editing the maps:

atmosphere.script	- lighting, fog, etc.
content.script	- file objects of the mission, groups of objects, etc.
hmap.raw	- Landscape cards
hwater.raw	- the water level on the map
mission.script	- main mission file, control scripts, etc.
missiontasks.script	- scripts management units \ groups.
routerzone.bmp	- road map of the landscape.
terrainzone.bmp	- map of vegetation, roads, surface.
micro.bmp	- map microtexture landscape, the location microtexture tiles on the landscape.
positionwatchers.script	- scripts recursive call monitoring changes in the location of interactive objects of the mission (units, buildings, navpoints ...)
worldmatricies.script	- classes of the landscape, an indication of the used texture, etc.
terrain.script	- parameters of the landscape of the mission, configuration of the forest.
Ind.tex	- the texture of the landscape minimum resolution.(Some files have a number in the name of the mission, this list does not specify numbers, so as not to create confusion)The remaining files located in the directory of the mission are also used in the mission, butnot directly related to editing maps.
lensflare.script	- non-binding effect of illumination.
forest.tex	- texture of remote forests, the same for all missions.

Editing landscape



Editing the landscape is made using the menu Terraform, the tool has a diverse set of brushes that are used to change the landscape maps.

Adjustment of the brushes is carried out with the following keys

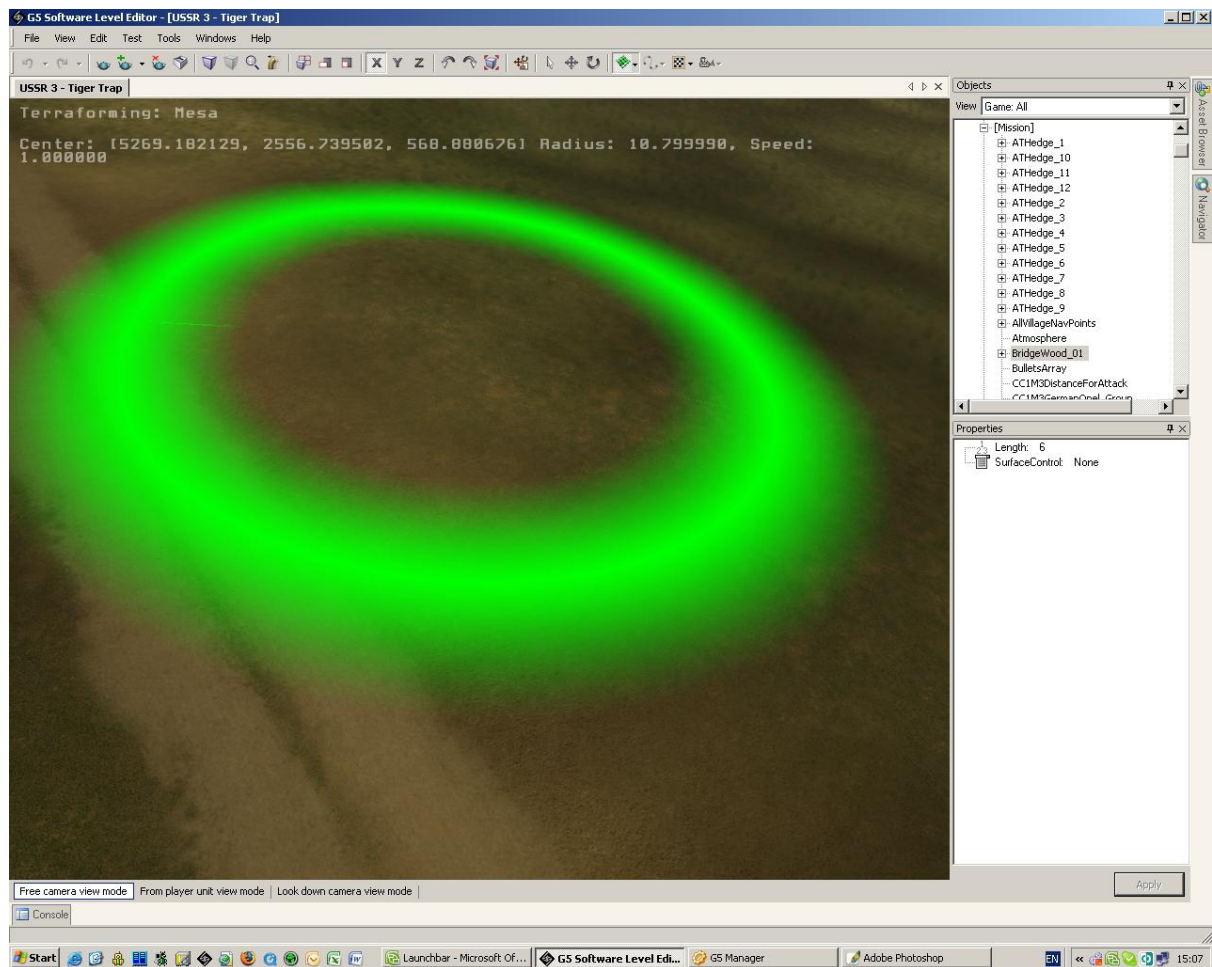
<+> - Increase the size of brushes.

<-> - Reduce the size of brushes.

<>> - Increase the intensity of changes.

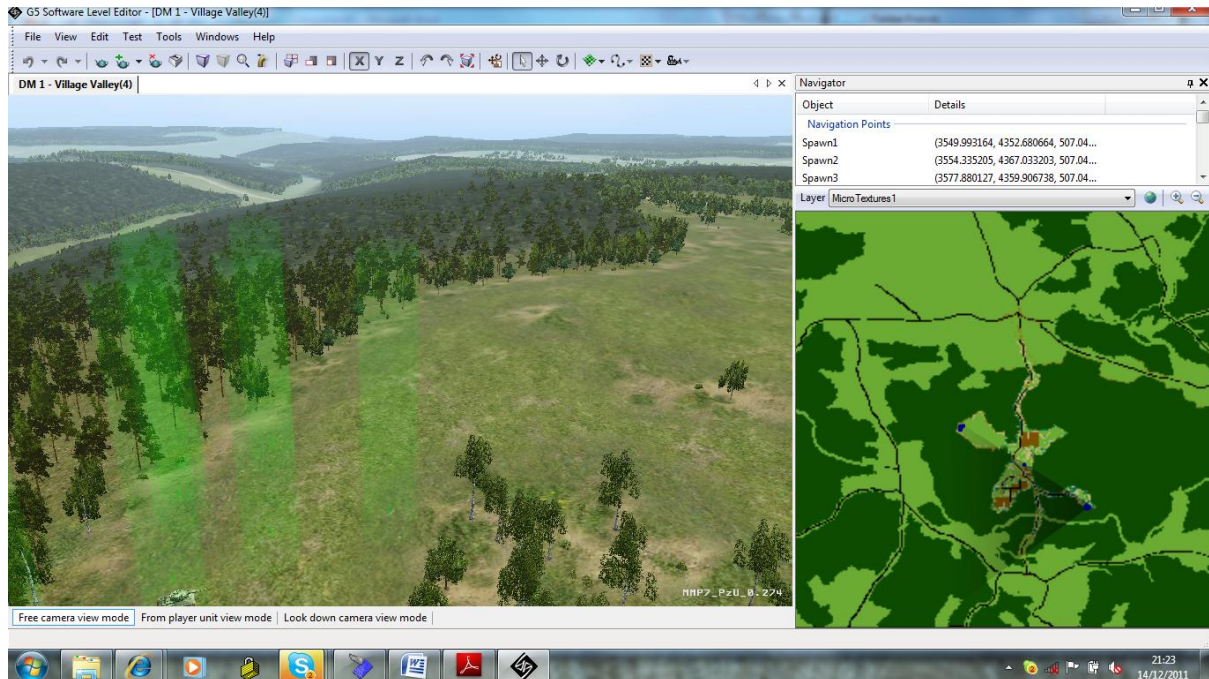
<<> - Reduce the intensity of changes.

When using the brush, you will see at the top of the camera a displayed line with the parameters of the current brush.



Save edited work using the menu bar File> Save Level. Terrain maps will be saved in a file in the directory hmap.raw mission. (The file must be writable).

Navigator



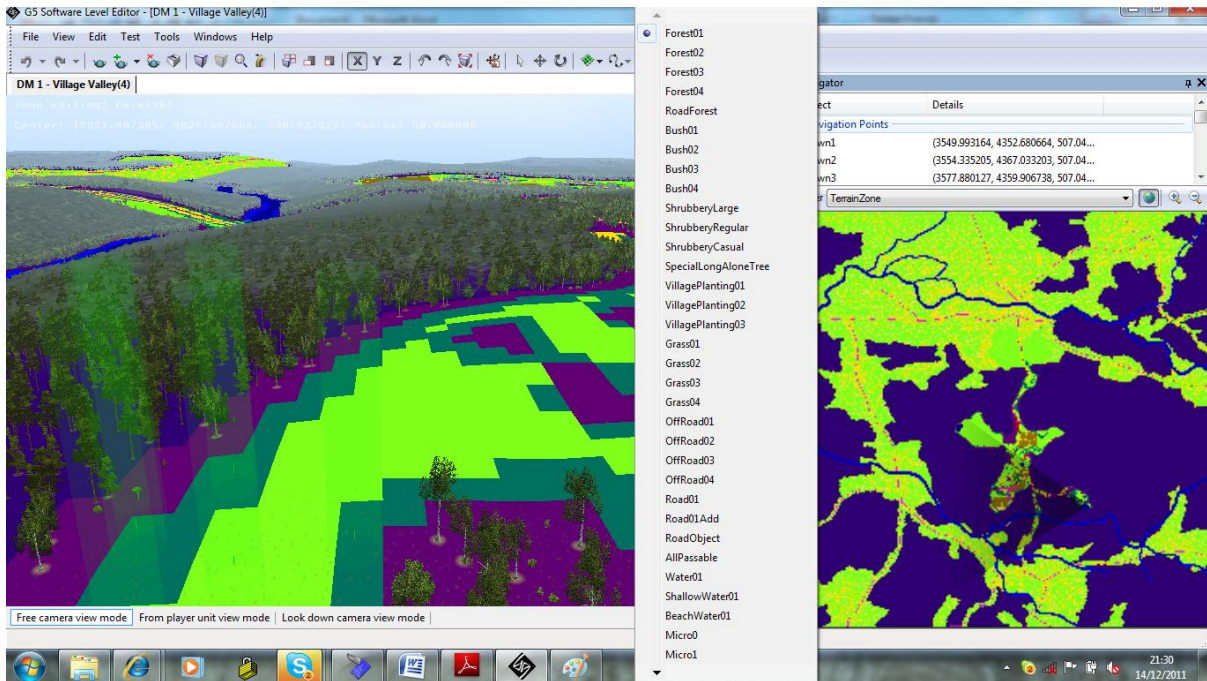
On the right hand side of the Editor window there is a tab called Navigator, which opens the additional toolbar, click in it to control the movement of the camera on the map. Logic navigator is similar use of the logic of the minimap window - click with the LMB to move the camera to the point of the click, the camera is shown on the minimap window triangle, an acute angle which is to point the camera on the landscape. One of the most popular and useful features is the ability to show the browser on the landscape layout of the following layers:

Routerzonemap - road map of the landscape.

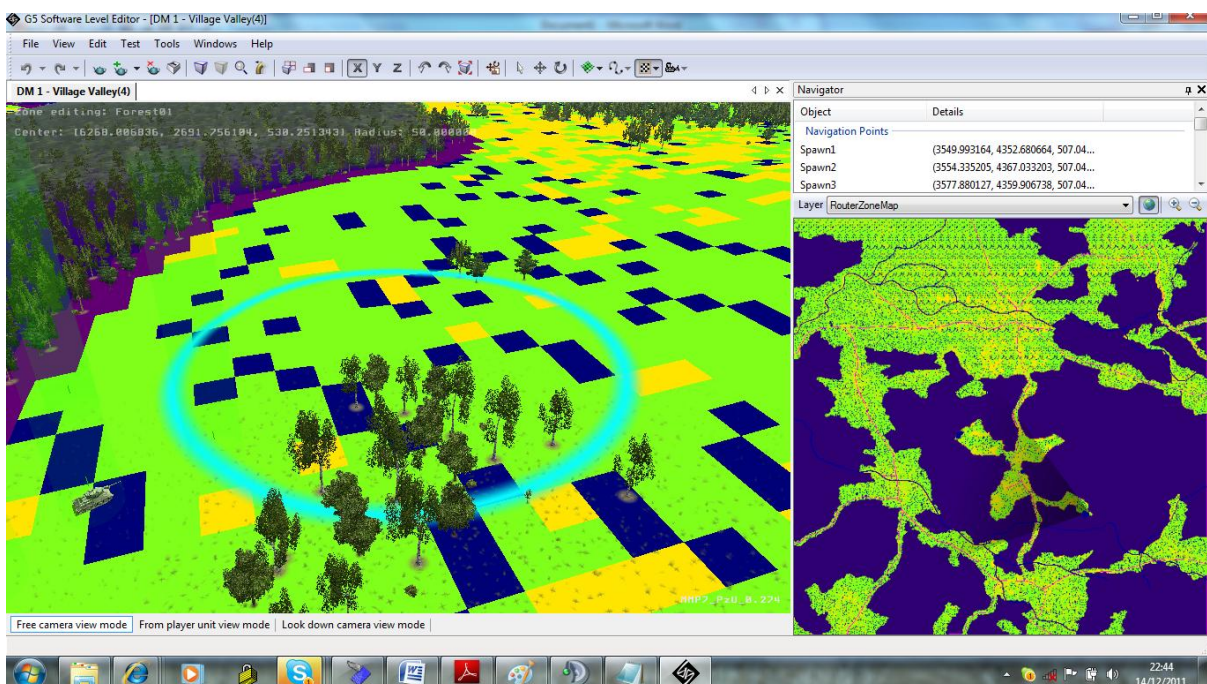
Terrainzone - map of vegetation, roads, surface.

microtextures - map microtexture landscape, the location microtexture tiles on the landscape. If the combo box in your browser to select the appropriate layer, and then click on the button with a stylized image of a globe, then the specified layer will be shown on the main form of the camera.

Editing vegetation



First pin the navigator to the window and using the drop down box select terrain zone Click on the zone editor drop down on the top menu bar one Editor and you will see that 19 of the first twenty entries relate to vegetation types. You will need to click on the world icon in navigator to show the vegetation zones in the main window as shown above.



The zone editing tool has a diverse set of brushes that are used to change the vegetation.

Adjustment of the brushes is carried out with the following keys

<+> - Increase the size of brushes.

<-> - Reduce the size of brushes.

When using the brush, you will see at the top of the camera a displayed line with the parameters of the current brush.

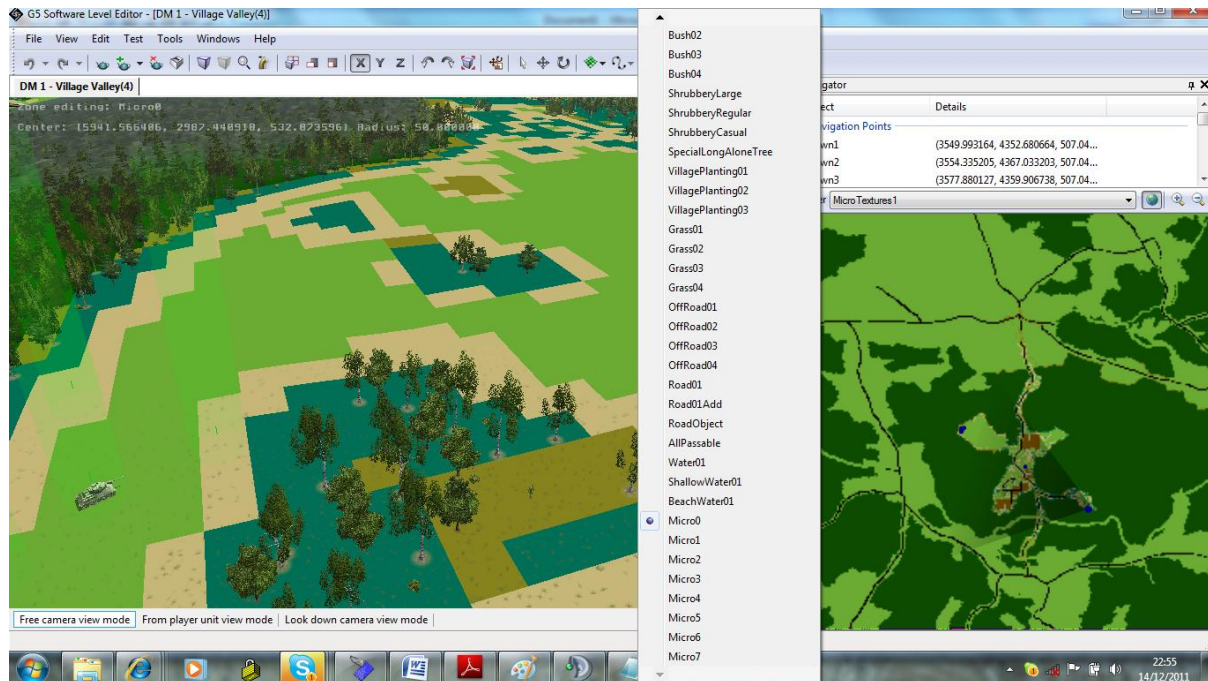
terrainzone.bmp - map of vegetation, roads, surface. file contains this information about, what tiles are placed on the surface in the engine, you can see the types of vegetation, colour index in use in the navigator, main editor screen and edit them.

This is achieved in the in the editor by using a brush from the menu ZoneEditor. Overwrite the tile by placing the brush in the desired location and left clicking with the mouse, however since editor is not good at real time tracking of changes to vegetation, you must save and restart the mission to see what changes have been made.

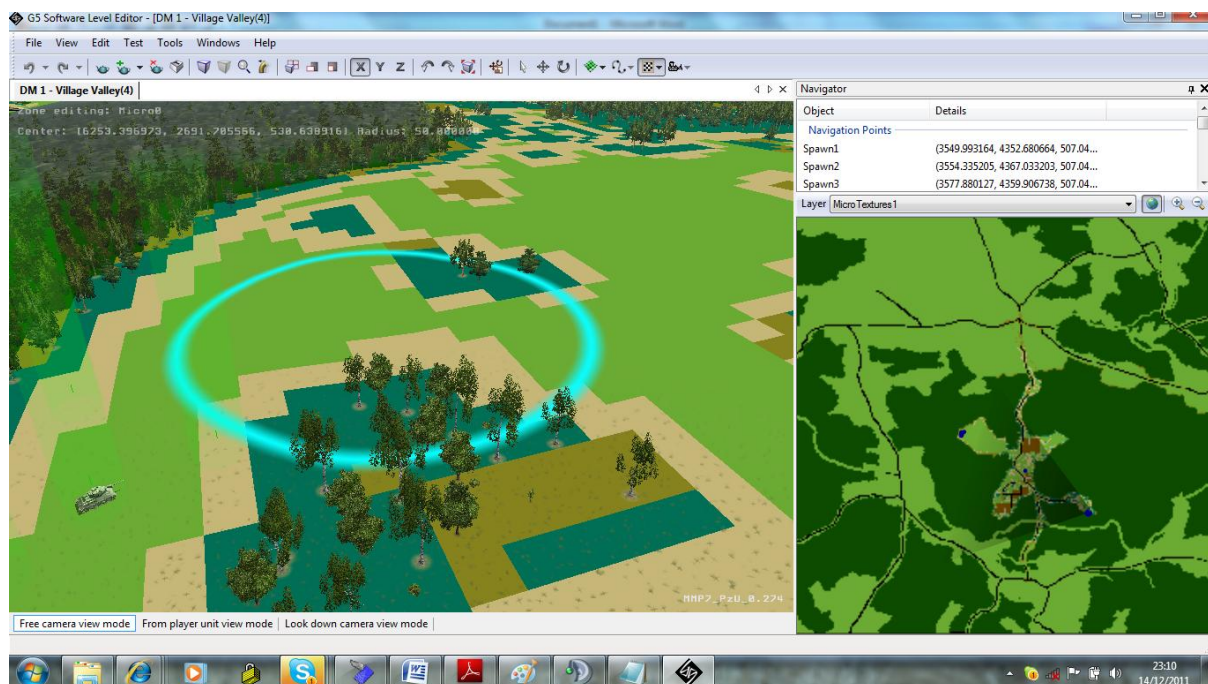
Perhaps for this reason it is more convenient to edit the map vegetation using a graphical editor such as Adobe Photoshop, using the instrument of choice and colours from an image (because vegetation indices are fixed and should not matched in colour and have a digital-matched with the correct Hex Colour code and or RGB code). To view the results of work you will need to restart the mission in the editor.

Zone Editor Classes	Colour Code Hex	Colour Code RGB	Colour Swatch	In editor view	
forest01	#ffcc00	R:255 , G:204 , B:0			
forest02	#ff99ff	R:255 , G:153 , B:255			
forest03	#ff99cc	R:255 , G:153 , B:204			

Editing microtexture



First pin the navigator to the window and using the drop down box select Micro Textures 1. Click on the zone editor drop down on the top menu bar one Editor and you will see that the last entries relate to micro texture types. You will need to click on the world icon in navigator to show the microtexture zones in the main window as shown above.



The zone editing tool has a diverse set of brushes that are used to change the microtextures.

Adjustment of the brushes is carried out with the following keys

<+> - Increase the size of brushes.

<-> - Reduce the size of brushes.

When using the brush, you will see at the top of the camera a displayed line with the parameters of the current brush.

Micro.bmp - map of microtextures, underlying textures that make up the base surface textures. The file contains this information about, what tiles are placed on the surface in the engine, you can see the types of microtextures, colour index in use in the navigator, main editor screen and edit them.

This is achieved in the in the editor by using a brush from the menu ZoneEditor. Overwrite the tile by placing the brush in the desired location and left clicking with the mouse, however since editor is not good at real time tracking of changes to microtextures, you must save and restart the mission to see what changes have been made.

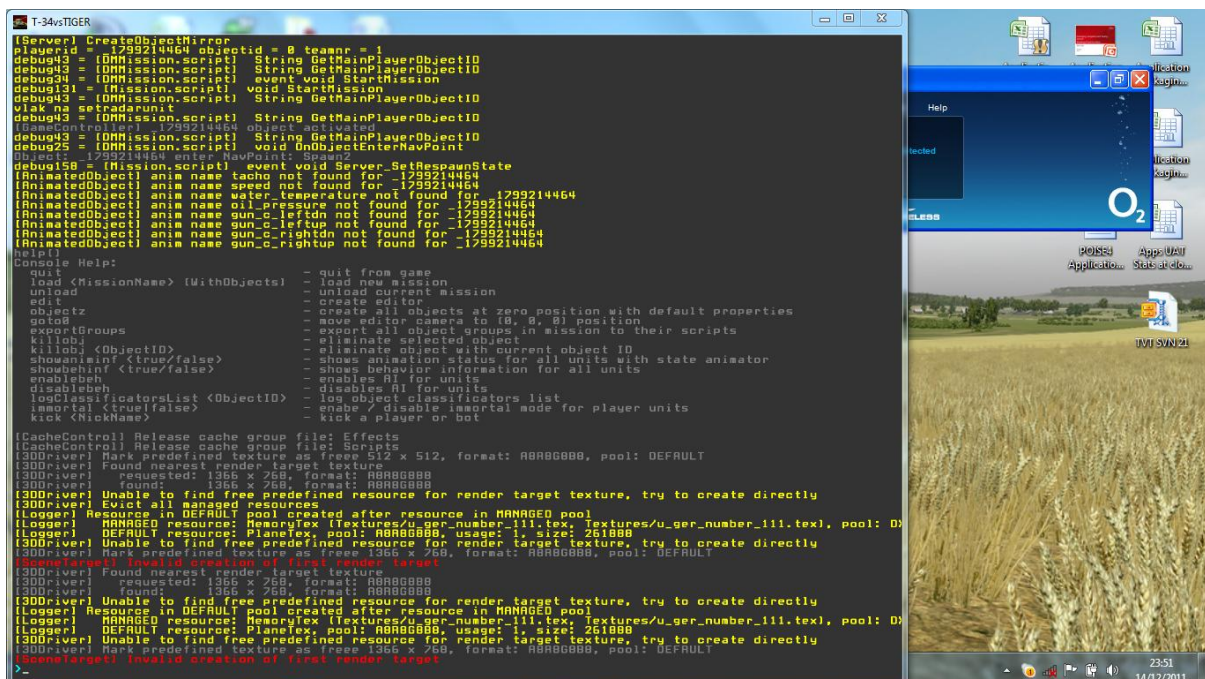
Perhaps for this reason it is more convenient to edit the map microtextures using a graphical editor such as Adobe Photoshop, using the instrument of choice and colours from an image (because microtextures indices are fixed and should not matched in colour and have a digital-matched with the correct Hex Colour code and or RGB code). To view the results of work you will need to restart the mission in the editor.

Log and Console

To unlock the console (will be available in the editor when you press F3 or F7, depending on the camera mode) in the file \ Scripts \ gamesettings.script EnableConsole set the variable to true.

```
final static boolean EnableConsole = true;
```

The console allows you to view code of the mission during its implementation and use the command line. Use the help command for a complete list of commands and their format only running the game outside the editor.



Ivan's Scenario Tutorial

Lesson 1. Create your own mission.

Author: Spogreev Ivan. Head of Sonar Games.

In this tutorial I will try to tell you in detail how to make your own mission for T34vsTiger. To do this we need to run the game and editor for the game.

Introduction.

The game editor in T34vsTiger is a pretty powerful tool. It allows you to view game objects, models, effects and animation. In it we can create / edit levels and immediately play them, we can run any mission and edit them on the fly.

The entire game is represented in the scripts. Scripts have the extension *.script and have a Java like language. The main scripts are in the folder Scripts, they described the entire game logic and the basic behaviour of objects, etc. The same is true for each mission, having its own scripts characterizing it. They are in their respective folders missions.

For example, the first mission for Russia would be in

"Missions \ Campaign_1 \ Mission_1".

In these scripts are described the arrangement of objects on the level, the conditions of failure and success of the mission and AI behaviour for the units. So the first thing that we need to do to create our own mission is to create separate folder for it.

Step 1. Separate folder for the mission

Create a new folder for our mission call it Mission1. In principle, it does not matter where you create the folder but for generality, let's create it in the folder Missions. And since we may have a lot of missions, use another shared folder (MyMission) to set up our levels.

Thus, you should see a new path:

Mission \ MyMission \ Mission1.

Next you must create all necessary files. We simply copy the contents from the test mission, which you will find at "Missions \ MISSIONS \ TESTMISSION". Copy all the files into our folder. This will be blank for further changes.

Step 2. Edit the scripts of the mission.

In the scripts we describe classes of objects. We can not allow duplicate class names to occur. We now have just such a situation in that we have copied files.

We need to modify them and rename the classes.

The list of files that need to be changed are:

Atmosphere.script
Content.script
Mission.script
MissionTasks.script
MissionTestStrings.script
Terrain.script
WorldMatricies.script

As we see, all the files with the extension *. script.

We will make changes in the following order.

First, edit the **atmosphere.script**. This sets the lighting setup, fog, shadows, etc. for the level. If we look at what it described class **TestAtmosphere**. We should rename the class to a name of our own, that is specific for this particular mission.

We will call it **Mission1Atmosphere**.

<pre>class <u>TestAtmosphere</u> extends CCommonAtmosphere { boolean IsLightEnabled = true; boolean IsAntiSunEnabled = true; Vector SunDirection = new Vector(-0.051952, -0.155542, -0.155348); Color AmbientLight = new Color(47.0 / 255.0, 47.0 / 255.0, 62.0 / 255.0); Color SunColor = new Color(239.0 / 255.0, 236.0 / 255.0, 231.0 / 255.0); Color AntiSunColor = new Color(162.0 / 255.0, 174.0 / 255.0, 185.0 / 255.0); float DistanceToSun = 2000.0;</pre>	<pre>class <u>Mission1Atmosphere</u> extends CCommonAtmosphere { boolean IsLightEnabled = true; boolean IsAntiSunEnabled = true; Vector SunDirection = new Vector(-0.051952, -0.155542, -0.155348); Color AmbientLight = new Color(47.0 / 255.0, 47.0 / 255.0, 62.0 / 255.0); Color SunColor = new Color(239.0 / 255.0, 236.0 / 255.0, 231.0 / 255.0); Color AntiSunColor = new Color(162.0 / 255.0, 174.0 / 255.0, 185.0 / 255.0); float DistanceToSun = 2000.0;</pre>
--	--

Next, make changes to the **Content.script**. This script contains a description of the objects on the level, their position, property, etc. It is generated automatically when you save level.

Rename the class name, file path and name of the class as a string.

<pre>class <u>TestContent</u> { static String m_ObjectListClassFile = "<u>Missions/MISSIONS/TestMission/Content.sc</u> <u>ript</u>"; static String m_ObjectListClassName = "<u>TestContent</u>"; static Array m_MissionObjectList = [["Atmosphere", ..., ..., new Matrix(1.000000, 0.000000, 0.000000, 0.000000, 0.000000, 1.000000, 0.000000, 0.000000, 0.000000, 0.000000, 1.000000, 0.000000, 0.000000, 0.000000, 0.000000, 1.000000), ...], ...];</pre>	<pre>class <u>Mission1Content</u> { static String m_ObjectListClassFile = "<u>Missions/MyMission/Mission1/Content.scri</u> <u>pt</u>"; static String m_ObjectListClassName = "<u>Mission1Content</u>"; static Array m_MissionObjectList = [["Atmosphere", ..., ..., new Matrix(1.000000, 0.000000, 0.000000, 0.000000, 0.000000, 1.000000, 0.000000, 0.000000, 0.000000, 0.000000, 1.000000, 0.000000, 0.000000, 0.000000, 0.000000, 1.000000), ...], ...];</pre>
---	---

In addition, you must remove all the objects that we do not require that were created to test the mission, which we took as a model. We should be left only with the names of objects:

Atmosphere

MainPlayerUnit

Terrain

CrossRoad2_micro (*You can remove it, but it will be left in my folder at present*)

The name of the object - this is the first parameter in an object record (understand, when you find appropriate names). Deleting everything else, you should receive the following picture (do not forget to remove the comma after recording

CrossRoad2_micro):

** not sure on this one, I left **CrossRoad2_micro** in my folder*

```

class MissionContent
{
    static String m_ObjectListClassFile = "Missions/MyMission/Mission1/content.script";
    static String m_ObjectListClassName = "MissionContent";

    static Array m_MissionObjectList = [
        [
            "Atmosphere",
            "...",
            "...",
            new Matrix(
                1.000000, 0.000000, 0.000000, 0.000000,
                0.000000, 1.000000, 0.000000, 0.000000,
                0.000000, 0.000000, 1.000000, 0.000000,
                0.000000, 0.000000, 0.000000, 1.000000
            ),
            [
                ["FogColorXPos", new Color(0.815686, 0.850980, 0.882353, 1.000000)],
                ["FogColorXNeg", new Color(0.815686, 0.850980, 0.882353, 1.000000)],
                ["FogColorYPos", new Color(0.815686, 0.850980, 0.882353, 1.000000)],
                ["FogColorYNeg", new Color(0.815686, 0.850980, 0.882353, 1.000000)],
                ["FogNear", 10.000000],
                ["FogFar", 800.000000],
                ["FogFarMax", 3500.000000],
                ["EnableHorizontAdjustment", true],
                ["SunColor", new Color(0.937255, 0.925490, 0.905882, 1.000000)],
                ["SunIntensity", 1.000000],
                ["AntiSunColor", new Color(0.811765, 0.819608, 0.854902, 1.000000)],
                ["AntiSunIntensity", 0.300000],
                ["AntiSunAngle", 10.000000],
                ["SunDirection", new Vector(-0.005952, -0.155542, -0.305348)],
                ["AmbientLight", new Color(0.200000, 0.200000, 0.200000, 1.000000)],
                ["ShadowColor", new Color(0.400000, 0.400000, 0.400000, 1.000000)],
                ["FogMode", "Exp"],
                ["FogDensity", 0.000300],
                ["TreeLightKoeff", 0.000000],
                ["TreeNormalRelax", 0.000000]
            ]
        ],
        [
            "MainPlayerUnit",
            "GameObject",
            "CTankT34_85_44Unit",
            new Matrix(
                -0.977255, -0.212067, 0.000000, 3822.968994,
                0.212067, -0.977255, 0.000000, 6137.772949,
                0.000000, 0.000000, 1.000000, 608.984070,
                0.000000, 0.000000, 0.000000, 1.000000
            ),
            [
                ["IsPlayer", true],
                ["IsManual", true],
                ["Affiliation", "FRIEND"],
                ["SurfaceControl", "None"]
            ]
        ],
        [
            "Terrain",
            "...",
            "...",
            new Matrix(
                1.000000, 0.000000, 0.000000, 0.000000,
                0.000000, 1.000000, 0.000000, 0.000000,
                0.000000, 0.000000, 1.000000, 0.000000,
                0.000000, 0.000000, 0.000000, 1.000000
            ),
            [
                ["ForestTexturescale", 0.500000],
                ["ForestShadowBorderscale", 0.400000],
                ["ForestShadowAlphaScale", 0.500000],
                ["ForestBeginFadeDist", [250.000000, 480.000000]],
                ["VertForestDistMin", 800.000000],
                ["VertForestDistMax", 8000.000000],
                ["VertForestAngleMin", 0.000000],
                ["VertForestAngleMax", 45.000000],
                ["VertForestBeginFadeDist", [250.000000, 410.000000]],
                ["TerrainShadowPower", 1.000000],
                ["IsAntiLighting", true],
                ["WaterFresnelPower", 2.000000]
            ]
        ],
        [
            "CrossRoad2_micro",
            "Locator",
            "CTerrainPatch",
            new Matrix(
                0.011889, -0.999929, 0.000000, 5185.122070,
                0.999929, 0.011889, 0.000000, 2949.116211,
                0.000000, 0.000000, 1.000000, 586.236694,
                0.000000, 0.000000, 0.000000, 1.000000
            ),
            [
                ["Dimensions", new Vector(15.000000, 15.000000, 0.350586)],
                ["TextureName", "Textures/CrossRoad_micro.tex"],
                ["ZoneName", "Textures/CrossRoad_zone.bmp"],
                ["Bordersize", 2.000000]
            ]
        ]
    ]
};

static Array m_MissionCameraScriptList = [
    [
        0.000000,
        "KEY_START",
        [
            ["CinemaId", "Default"],
            []
        ]
    ]
];

```

The next action will remove all entries in **MissionTasks.script**. This script is responsible for Task of the units. Task - Model the behaviour of AI. Creating their own problem, we can control units, giving them different commands depending on situations. In this mission, we do not need these properties, especially Tasks, so we can safely clean the file.

```
class tmp_AttackTigers extends CBaseUnitGroup
{
    Array KillList = [
        "tmp_GermanTiger1",
        "tmp_GermanTiger2",
        "tmp_GermanT4_1",
        "MainPlayerUnit"
    ];

    void Init()
    {
        CBaseUnitGroup::Init();
        logWarning("TSovietKillGroup void Init");
        SetEnemyReactionType(ERT_AGGRESSIVE);
        ActivateRadar(true);
        ActivateFire(true);
    }
}

class testresst extends CC1M1Gr_BaseInfantry
{
    float m_InfantrySpeed = 2.00;

    void OnPathEndReached()
    {
        CC1M1Gr_BaseInfantry::Brighten(m_StartPosition);
    }
}

// Left Infantry One Group
class testresst2 extends testresst
{
    String m_PointAttack = "SoldNP2";
}
```

Scripts **MissionTestStrings.script**, **Terrain.script** **WorldMatricies.script** are not particularly significant for us, so I will not describe them now, I will show screen shots of the scripts with code lines that need changing underlined in red:

Terrain.script

<pre> class CTestTerrain extends CBaseTerrain, CBaseZoneMap { CTestTerrain() { if (!CDebugSettings::LoadWater) waterRegions = []; } void CreateForesRegions() { Component Materials = new #MaterialManager<CSTBaseForestCLSkin>(); RegisterForestRegion([ZMC_Forest01], Materials, "Missions\\missions\\TestMission\\Forest_Test.tex", [L1, L2, L3, L3], [0.0, 8.0, 11.0, 13.0, 14.0], [500.0, 500.0, 1000.0, 1500.0, 2000.0], // min distance, start fade [1500.0, 1500.0, 2000.0, 2500.0, 2500.0] // max distance, hide layer); RegisterVerticalForest([ZMC_Forest01], "textures\\Foreststripe.tex", // number of rings of vertical forest [0.8], 17.0f); } CBaseTerrain::CreateForesRegions(); // terrain resources String SkinClass = "CTestTerrainskin"; // Sea level for compute air density float SeaLevel = 450.0; float BaseDensity = 0.125; // water regions description Array WaterRegions = [[new CWaterRegion(new Vector(23532.10, 29461.19, 1165.0), 1165.0)]; Array WaterMirrorMasks = [[CLASSIFICATOR_SHADOW, CLASSIFICATOR_UI, CLASSIFICATOR_TERRAINPATCH]]; float TerrainShadowPower = 10.5; boolean IsAntilighting = true; boolean IsPlanarWater = true; } class CTestTerrainskin extends CBaseTerrainskin { CTestTerrainskin() { SetupTerrainMainMaterial("Missions/MISSIONS/TestMission/Ind_Test.tex"); AppendMaterials(MissionSpecificMaterials); } Array MissionSpecificMaterials = []; } </pre>	<pre> class CMissionTerrain extends CBaseTerrain, CBaseZoneMap { CMissionTerrain() { if (!CDebugSettings::LoadWater) waterRegions = []; } void CreateForesRegions() { Component Materials = new #MaterialManager<CSTBaseForestCLSkin>(); RegisterForestRegion([ZMC_Forest01], Materials, "Missions\\MyMission\\Mission1\\Forest_Test.tex", [L1, L2, L3, L3], [0.0, 8.0, 11.0, 13.0, 14.0], [500.0, 500.0, 1000.0, 1500.0, 2000.0], // min distance, start fade [1500.0, 1500.0, 2000.0, 2500.0, 2500.0] // max distance, hide layer); RegisterVerticalForest([ZMC_Forest01], "textures\\Foreststripe.tex", // number of rings of vertical forest [0.8], 17.0f); } CBaseTerrain::CreateForesRegions(); // terrain resources String SkinClass = "CMissionTerrainskin"; // Sea level for compute air density float SeaLevel = 450.0; float BaseDensity = 0.125; // water regions description Array WaterRegions = [[new CWaterRegion(new Vector(23532.10, 29461.19, 1165.0), 1165.0)]; Array WaterMirrorMasks = [[CLASSIFICATOR_SHADOW, CLASSIFICATOR_UI, CLASSIFICATOR_TERRAINPATCH]]; float TerrainShadowPower = 10.5; boolean IsAntilighting = true; boolean IsPlanarWater = true; } class CMissionTerrainskin extends CBaseTerrainskin { CMissionTerrainskin() { SetupTerrainMainMaterial("Missions/MyMission/Mission1/Ind_Test.tex"); AppendMaterials(MissionSpecificMaterials); } Array MissionSpecificMaterials = []; } </pre>
---	--

MissionTestStrings.script

<pre> class CTestMissionStrings extends CCommonStrings { final static WString MissionName = getLocalized("MissionTest", "MissionName"); final static WString BriefingText = getLocalized("MissionTest", "BriefingText"); final static WString Objective01 = getLocalized("MissionTest", "Objective01"); final static WString Objective02 = getLocalized("MissionTest", "Objective02"); final static WString Objective03 = getLocalized("MissionTest", "Objective03"); } </pre>	<pre> class CMission1Strings extends CCommonStrings { final static WString MissionName = getLocalized("MissionTest", "MissionName"); final static WString BriefingText = getLocalized("MissionTest", "BriefingText"); final static WString Objective01 = getLocalized("MissionTest", "Objective01"); final static WString Objective02 = getLocalized("MissionTest", "Objective02"); final static WString Objective03 = getLocalized("MissionTest", "Objective03"); } </pre>
--	---

WorldMatricies.script

<pre> class <u>TestWorldMatrices</u> extends CWorldMatrices { final static float MatrixWidth = 9000.0; final static float MatrixHeight = 9000.0; } class <u>TestLandscapeLayer</u> extends CLandscapeLayer { final static int ImageWidth = 2049; final static int ImageHeight = 2049; final static String ImageFileName = <u>"Missions/MISSIONS/TestMission/hmap.raw";</u> final static float FloatValueFactor = 0.070000*257.0; } class <u>TestWaterHeights</u> extends CBaseWaterHeightsLayer { final static String ImageFileName = <u>"Missions/MISSIONS/TestMission/hwater.raw";</u> final static float FloatValueFactor = 0.070000*257.0; } class <u>TestTerrainZoneLayer</u> extends CTerrainZoneLayer, CBaseZoneMap { final static int ImageWidth = 1024; final static int ImageHeight = 1024; final static String ImageFileName = <u>"Missions/MISSIONS/TestMission/TerrainZone_Test.bmp";</u> } class <u>TestRouterZoneLayer</u> extends CTerrainZoneLayer, CBaseZoneMap { final static int ImageWidth = 2048; final static int ImageHeight = 2048; final static String ImageFileName = <u>"Missions/MISSIONS/TestMission/RouterZone_Test.bmp";</u> } class <u>TestMicroTextures1</u> extends CMicroTexLayer { final static String ImageFileName = <u>"Missions/MISSIONS/TestMission/micro_Test.bmp";</u> } </pre>	<pre> class <u>Mission1WorldMatrices</u> extends CWorldMatrices { final static float MatrixWidth = 9000.0; final static float MatrixHeight = 9000.0; } class <u>Mission1LandscapeLayer</u> extends CLandscapeLayer { final static int ImageWidth = 2049; final static int ImageHeight = 2049; final static String ImageFileName = <u>"Missions/MyMission/Mission1/hmap.raw";</u> final static float FloatValueFactor = 0.070000*257.0; } class <u>Mission1WaterHeights</u> extends CBaseWaterHeightsLayer { final static String ImageFileName = <u>"Missions/MyMission/Mission1/hwater.raw";</u> final static float FloatValueFactor = 0.070000*257.0; } class <u>Mission1TerrainZoneLayer</u> extends CTerrainZoneLayer, CBaseZoneMap { final static int ImageWidth = 1024; final static int ImageHeight = 1024; final static String ImageFileName = <u>"Missions/MyMission/Mission1/TerrainZone_Test.bmp";</u> } class <u>Mission1RouterZoneLayer</u> extends CTerrainZoneLayer, CBaseZoneMap { final static int ImageWidth = 2048; final static int ImageHeight = 2048; final static String ImageFileName = <u>"Missions/MyMission/Mission1/RouterZone_Test.bmp";</u> } class <u>Mission1MicroTextures1</u> extends CMicroTexLayer { final static String ImageFileName = <u>"Missions/MyMission/Mission1/micro_Test.bmp";</u> } </pre>
---	---

Next, we need to edit the main script of the mission itself, the file ***Mission.script***. Here we should change the class name of the mission and supporting classes it calls.

<pre> class CTestMission extends CSPMission { // // Construction and initialization // String n_LocalTime = "14:20:00"; String n_TerrainMapTextureName = "Textures/C1M3_Map.tex"; static String n_MissionBriefingPicMaterial = "C1M3BriefingPic"; Array n_MissionObjectives = []; boolean isDebug = true; Array n_NavpointsForPlayerMap = []; Float CockpitMapMinRange = 1000.0; Float CockpitMapMaxRange = 4500.0; int CockpitMapZoomSteps = 10; Vector MarksInitPoint = new Vector(1000.0, 1000.0, 0.0); Float CockpitMapNavNameMaxRange = 2000.0; Array CockpitMapAccessBox = [new Vector(2742, 1775.0, 0.0), new Vector(7242.0, 6275.0, 0.0)]; final static Array RouterWorkingZones = []; final static Array DefaultMask = [{"ENEMY", "MainMesh"}, {"NEUTRAL"}]; final static Array Mask = [{"ENEMY", "MainMesh"}, {"NEUTRAL", "InvisibleOnRadar"}]; boolean OnTarg = false; boolean TigerAggr = false; boolean MainPlayerStart = false; void CTestMission() { // Construct mission CSPMission("CTestMission", "TestContent"); // Set mission properties SetMissionTerrain(new #ChunkedTerrain<CTestTerrain>()); SetMissionAtmosphere(new #Atmosphere<TestAtmosphere>()); SetMissionSky(new #SkyObject<CSkyModel>()); if (CDebugSettings::LoadForest) SetMissionForest(new CSTBaseForestC1(GetMissionAtmosphere())); if (CDebugSettings::LoadRoads) SetMissionRoadsParms(new CBaseRoadC1()); if (CDebugSettings::LoadGrass) RegisterObject("Grass", new #Grass<CBaseGrassC1>()); SetMissionWorldMatrices(new #WorldMatrices<TestWorldMatrices>(), [[LAYER_TERRAIN_NAME, "TestLandscapeLayer"], [LAYER_TERRAIN_ZONE, "TestTerrainZoneLayer"], [LAYER_ROUTER_ZONE, "TestRouterZoneLayer"], [LAYER_MICROTEXTURE_MAP1, "TestMicroTextures1"], [LAYER_TERRAIN_WATERHEIGHTS, "TestWaterHeights"]]); SetRouterPrecalculatedGraph(new #RouterPrecalculatedGraph<CRouterPrecalculatedGraph>()); SetRouterMap("RouterMap_Layer1", new #RouterMap<CBaseRouterMap>(), 64, RouterWorkingZones); } </pre>	<pre> class CMission1Mission extends CSPMission { // // Construction and initialization // String n_LocalTime = "14:20:00"; String n_TerrainMapTextureName = "Textures/C1M3_Map.tex"; static String n_MissionBriefingPicMaterial = "C1M3BriefingPic"; Array n_MissionObjectives = []; boolean isDebug = true; Array n_NavpointsForPlayerMap = []; Float CockpitMapMinRange = 1000.0; Float CockpitMapMaxRange = 4500.0; int CockpitMapZoomSteps = 10; Vector MarksInitPoint = new Vector(1000.0, 1000.0, 0.0); Float CockpitMapNavNameMaxRange = 2000.0; Array CockpitMapAccessBox = [new Vector(2742, 1775.0, 0.0), new Vector(7242.0, 6275.0, 0.0)]; final static Array RouterWorkingZones = []; final static Array DefaultMask = [{"ENEMY", "MainMesh"}, {"NEUTRAL"}]; final static Array Mask = [{"ENEMY", "MainMesh"}, {"NEUTRAL", "InvisibleOnRadar"}]; boolean OnTarg = false; boolean TigerAggr = false; boolean MainPlayerStart = false; void CMission1Mission() { // Construct mission CSPMission("CMission1Mission", "Mission1Content"); // Set mission properties SetMissionTerrain(new #ChunkedTerrain<CMission1Terrain>()); SetMissionAtmosphere(new #Atmosphere<Mission1Atmosphere>()); SetMissionSky(new #SkyObject<CSkyModel>()); if (CDebugSettings::LoadForest) SetMissionForest(new CSTBaseForestC1(GetMissionAtmosphere())); if (CDebugSettings::LoadRoads) SetMissionRoadsParms(new CBaseRoadC1()); if (CDebugSettings::LoadGrass) RegisterObject("Grass", new #Grass<CBaseGrassC1>()); SetMissionWorldMatrices(new #WorldMatrices<Mission1WorldMatrices>(), [[LAYER_TERRAIN_NAME, "Mission1LandscapeLayer"], [LAYER_TERRAIN_ZONE, "Mission1TerrainZoneLayer"], [LAYER_ROUTER_ZONE, "Mission1RouterZoneLayer"], [LAYER_MICROTEXTURE_MAP1, "Mission1MicroTextures1"], [LAYER_TERRAIN_WATERHEIGHTS, "Mission1WaterHeights"]]); SetRouterPrecalculatedGraph(new #RouterPrecalculatedGraph<CRouterPrecalculatedGraph>()); SetRouterMap("RouterMap_Layer1", new #RouterMap<CBaseRouterMap>(), 64, RouterWorkingZones); } </pre>
--	--

After all the edits we can test our mission.

Step 3. The inclusion of our mission in the mission list.

In order to be able to download, edit and play our level, we need to add it to the list of missions that appear in the editor.

This list is in the script

lScripts\EditorMenuConfig.script, variable **MissionLoadList**. Find it and edit it by appending the following line:

["My Super Mission", "CMission1Mission"]

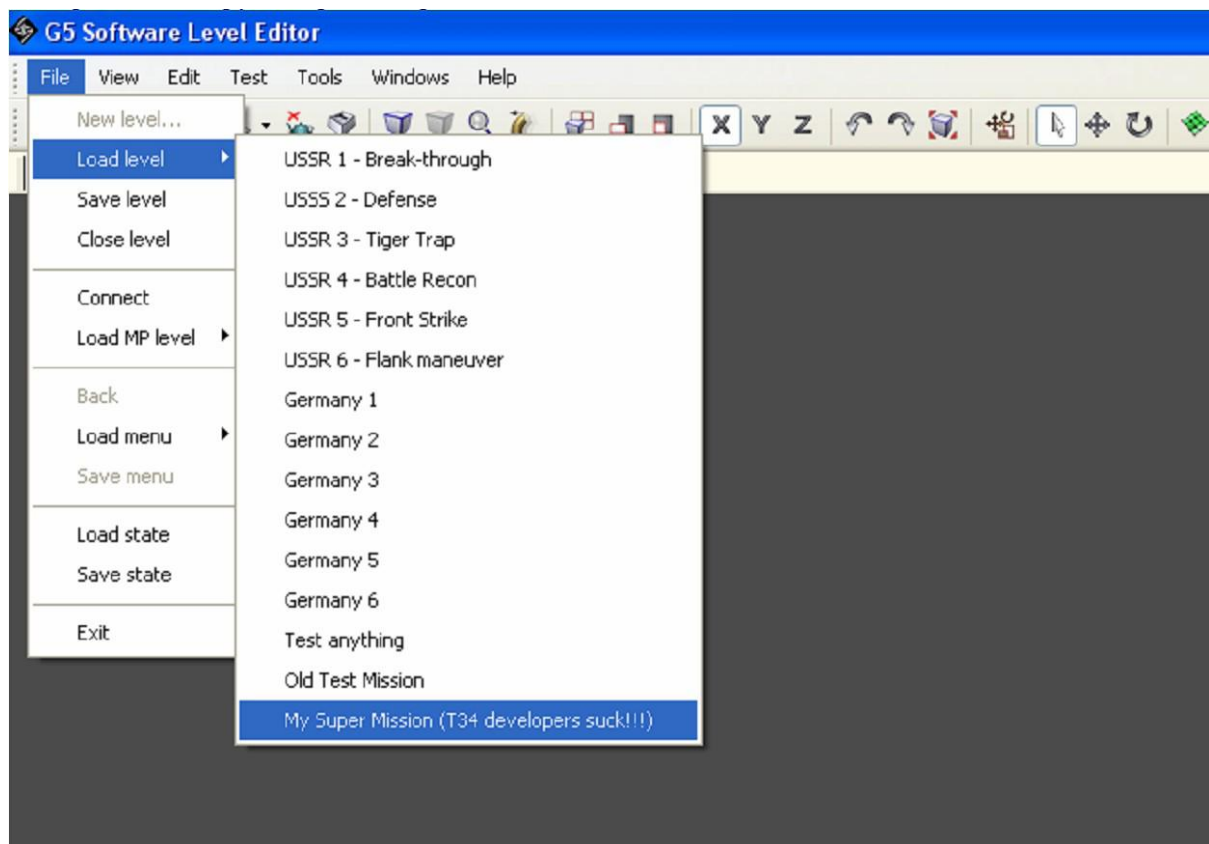
```

class CMenuConfig
{
    CMenuConfig()
    {
        FillZoneEditorTypeList();
    }

    //-----
    // List of missions in editor
    //-----
    final static Array MissionLoadList = [
        ["USSR 1 - Break-through", "CC1M1Mission"],
        ["USSS 2 - Defense", "CC1M2Mission"],
        ["USSR 3 - Tiger Trap", "CC1M3Mission"],
        ["USSR 4 - Battle Recon", "CC1M4Mission"],
        ["USSR 5 - Front Strike", "CC1M5Mission"],
        ["USSR 6 - Flank maneuver", "CC1M6Mission"],
        ["Germany 1", "CC2M1Mission"],
        ["Germany 2", "CC2M2Mission"],
        ["Germany 3", "CC2M3Mission"],
        ["Germany 4", "CC2M4Mission"],
        ["Germany 5", "CC2M5Mission"],
        ["Germany 6", "CC2M6Mission"],
        ["Test anything", "CTestMission"],
        ["Old Test Mission", "CMtMission"],
        ["My Super Mission (T34 developers suck!!!)", "CMission1Mission"]
    ];
}

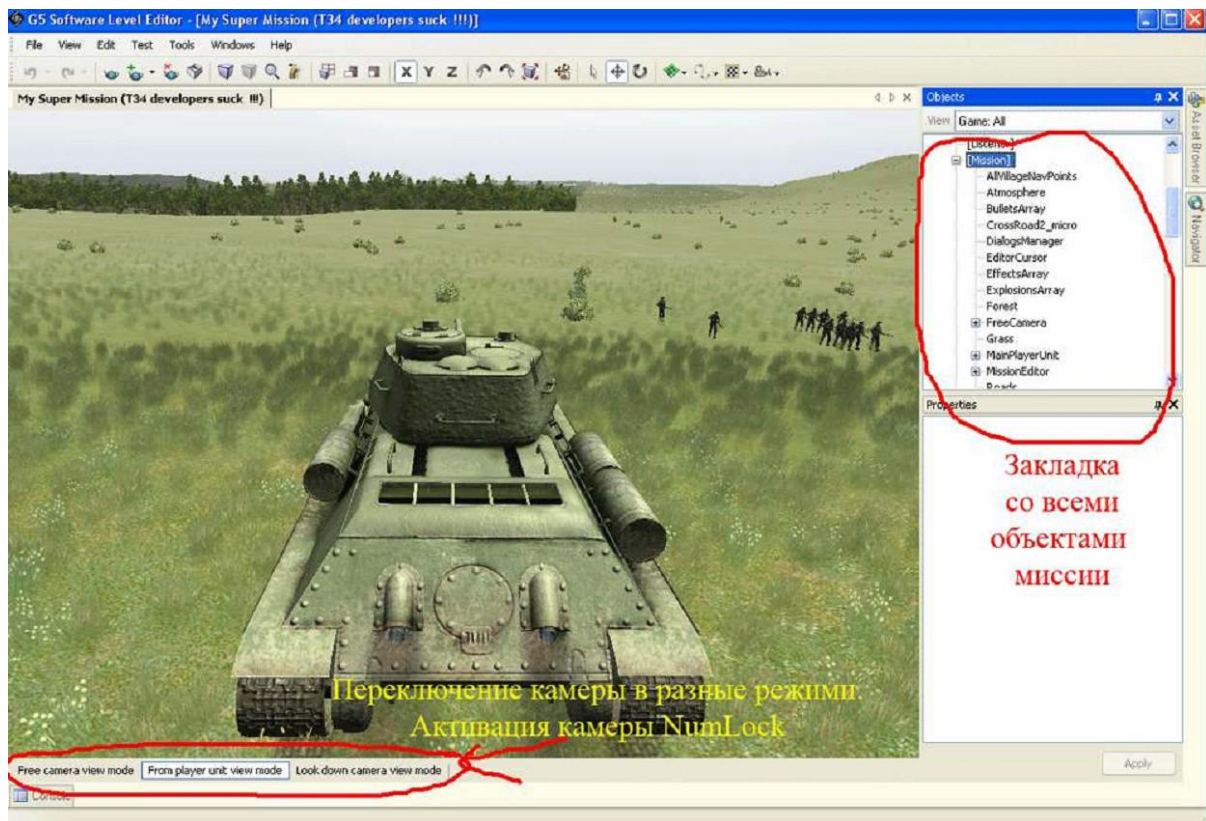
```

Now, if we start the editor, our mission will appear in the list.

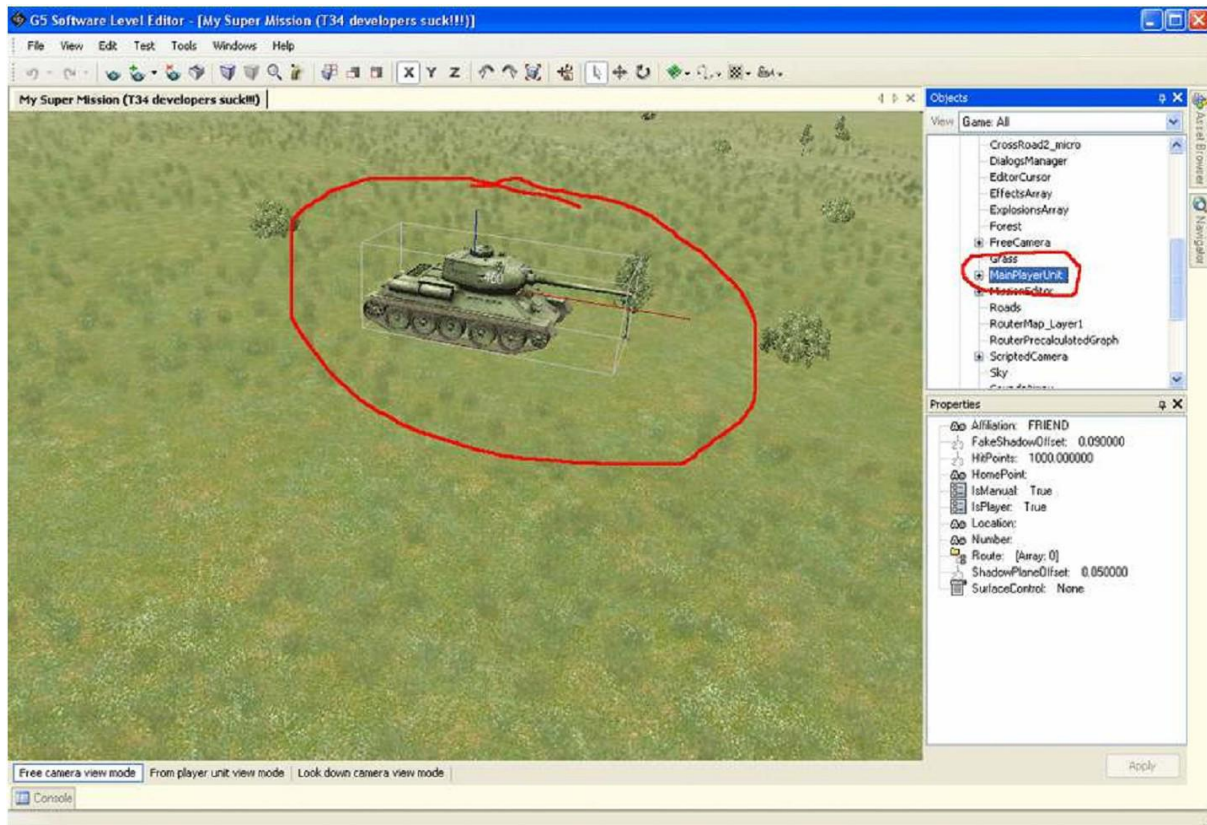


Step 4. Editor Basics.

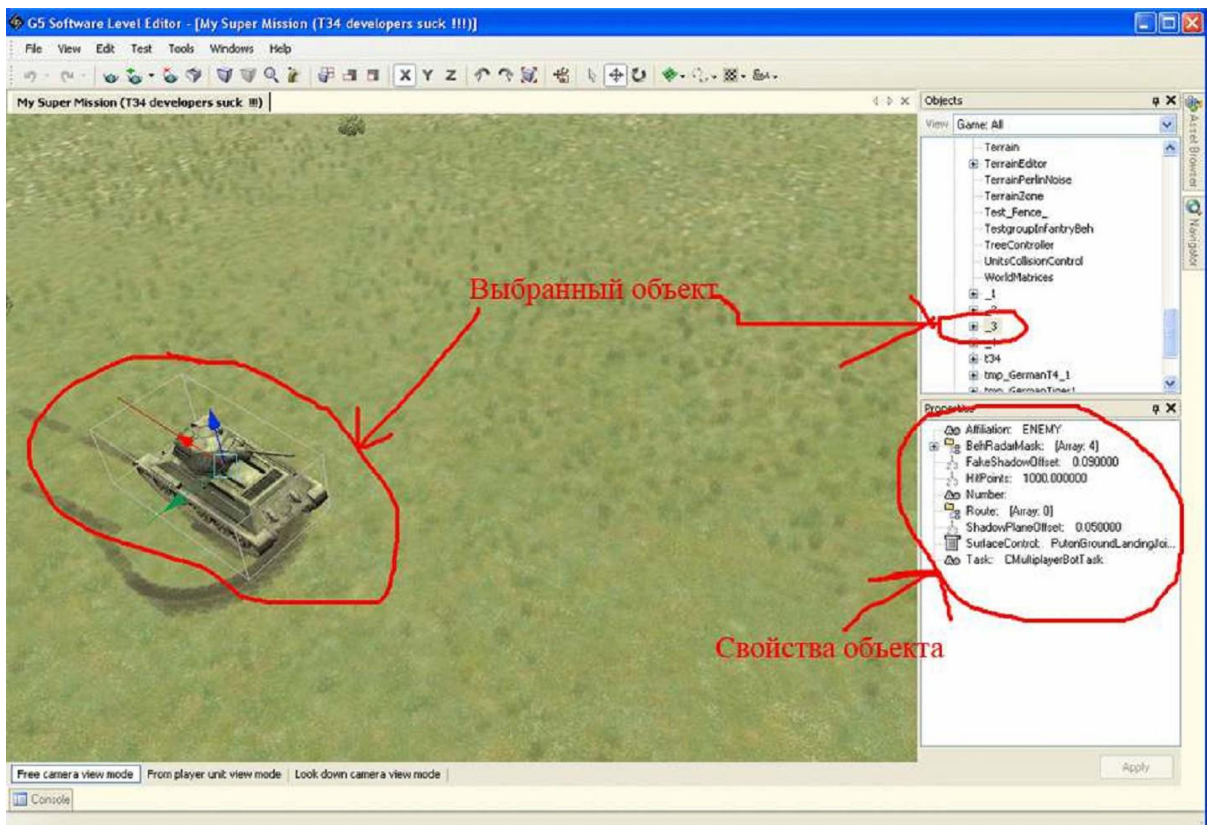
Open the mission and look at the screen. Located on the right is a list of objects in game. We are only interested in playing Mission, as there are all objects of our level.



We can also switch to different camera modes: Free, Player and Lookdown . When we activate the game camera, then we begin to play immediately. We can choose different objects directly from the objects list, or in a free camera to click the right mouse button next to an object (in this case algorithm for choosing this conditions is inaccurate, and may not always choose what you want).



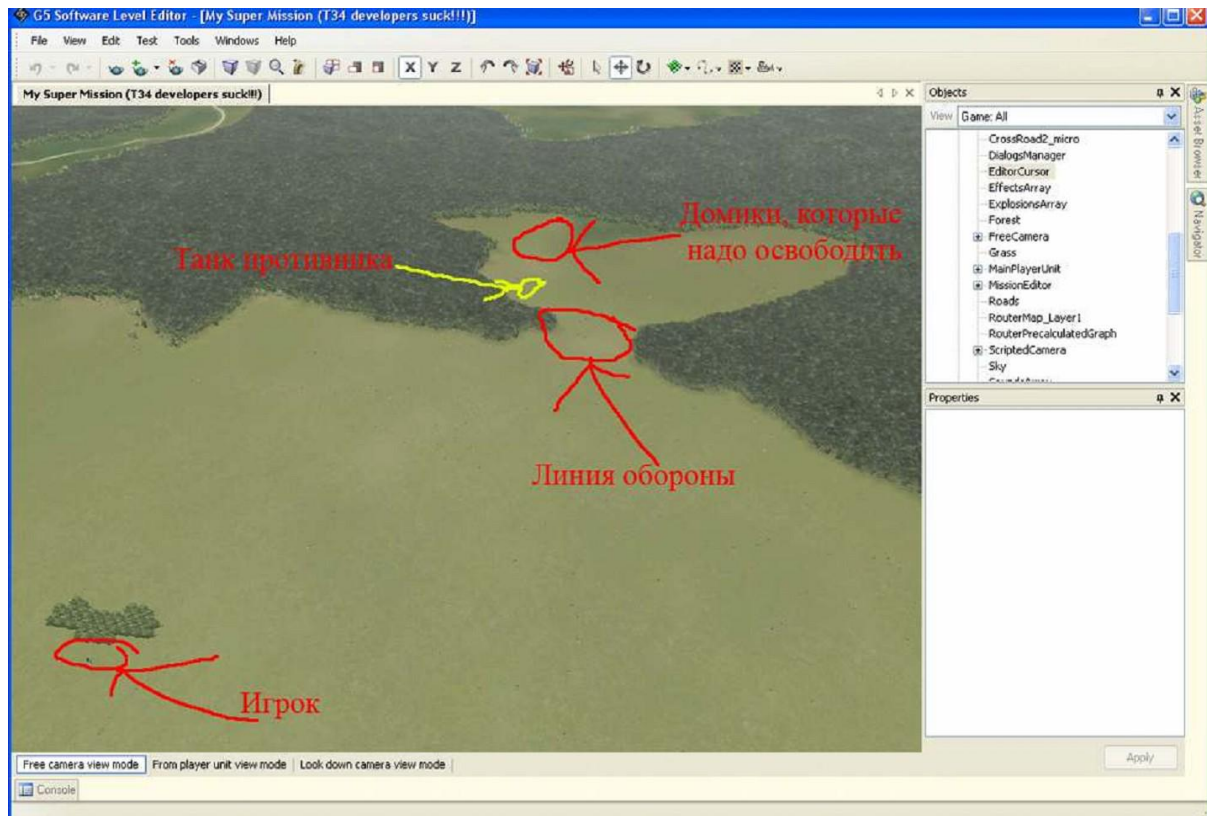
When we choose an object below it shows the properties of an object, they can be edited, added or deleted.



Properties for different objects / properties / icon and see what's what. Note that some times, if you do not click there, the editor may freeze.

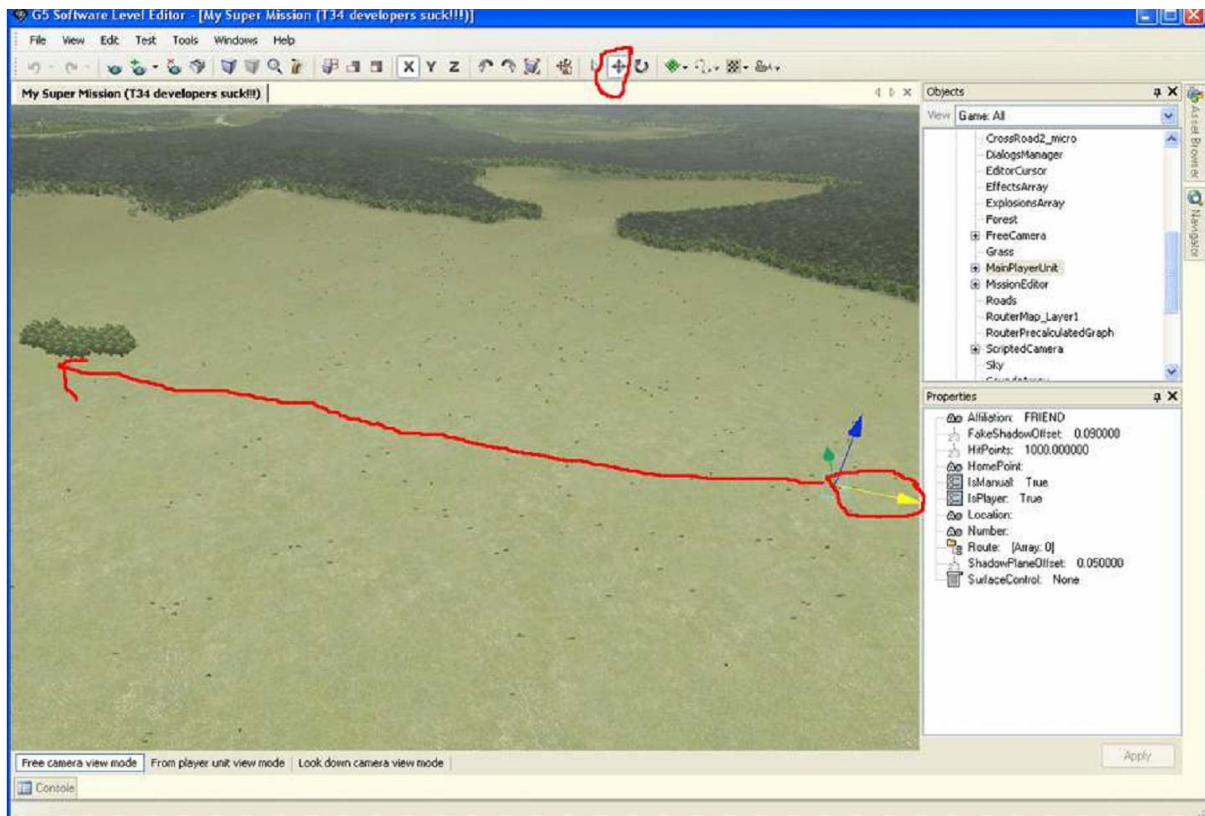
Step 5. The mission plan.

Created on our level we have one of our tanks. Let's decide what will be is the essence of the mission. I really have not bothered with a plot, so the plan will be simple - you need to move to a nearby village, breaking through the defensive line.



Step 6. Puts players in place.

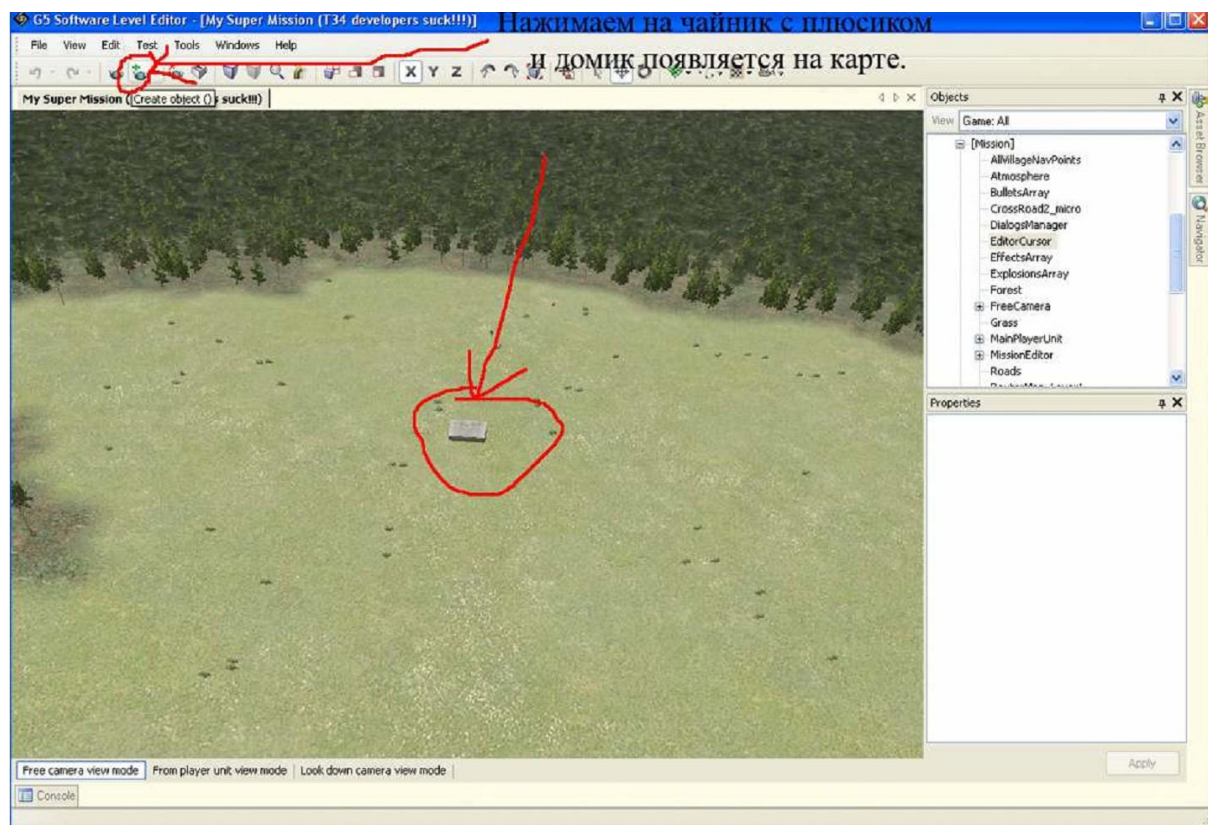
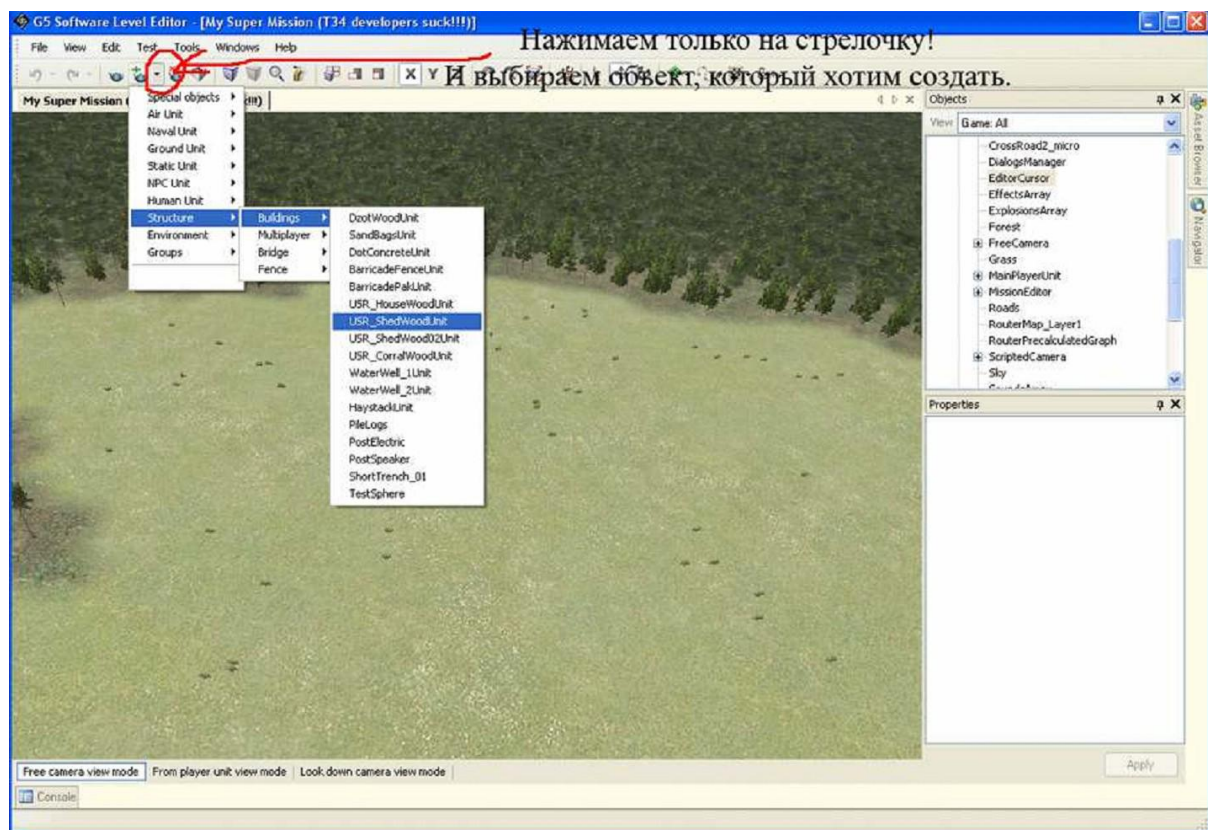
Because we created a level, the player is placed in the middle of a field rather than at the line, we can move it back by selecting the Move tool.



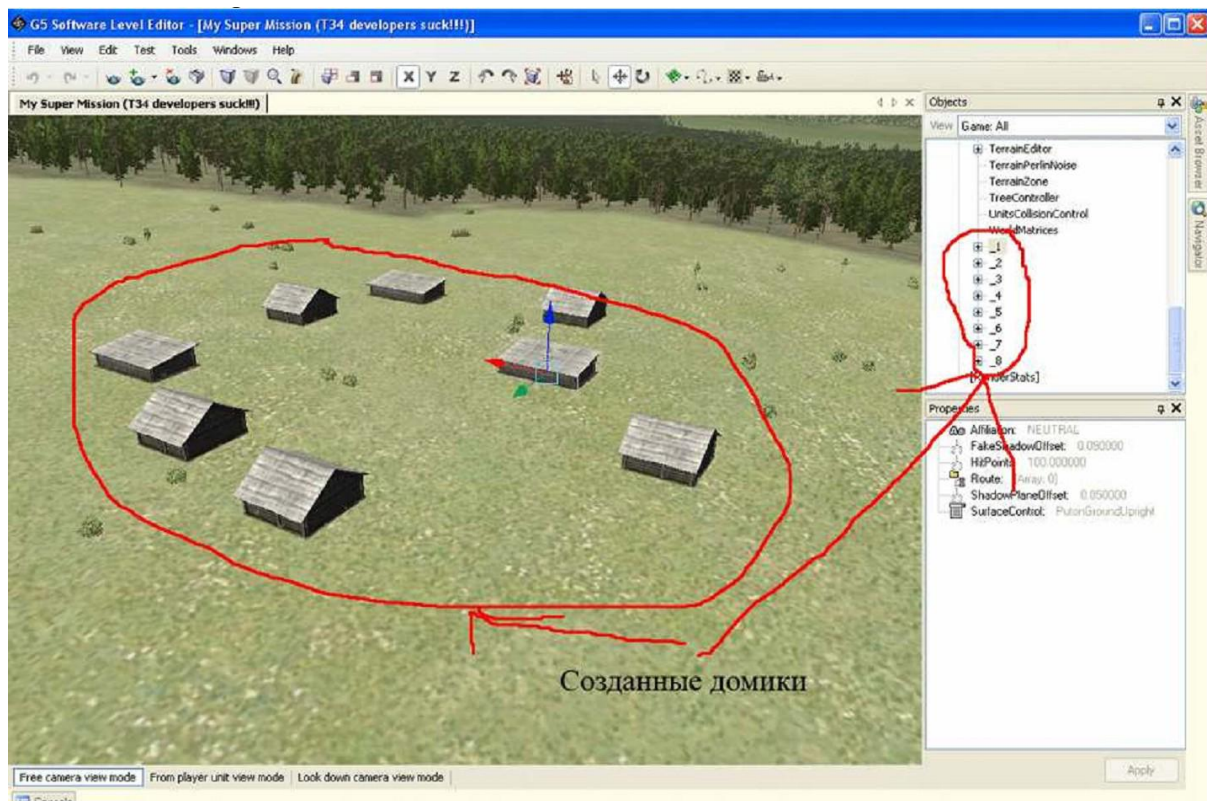
Attention! Due to the game physics, I do not recommend trying to drive the tank you moved ...Try it and see what happens ☺ Better save the level and load it again.

Step 7. Creation of the village.

Now create a few homes that will symbolize the captured village. Note the word "symbolize", because I do not claim to authenticity oh Belarusian villages. To create an object, we first need to choose its type, and then add it on the map. Screens illustrate the process of creating:

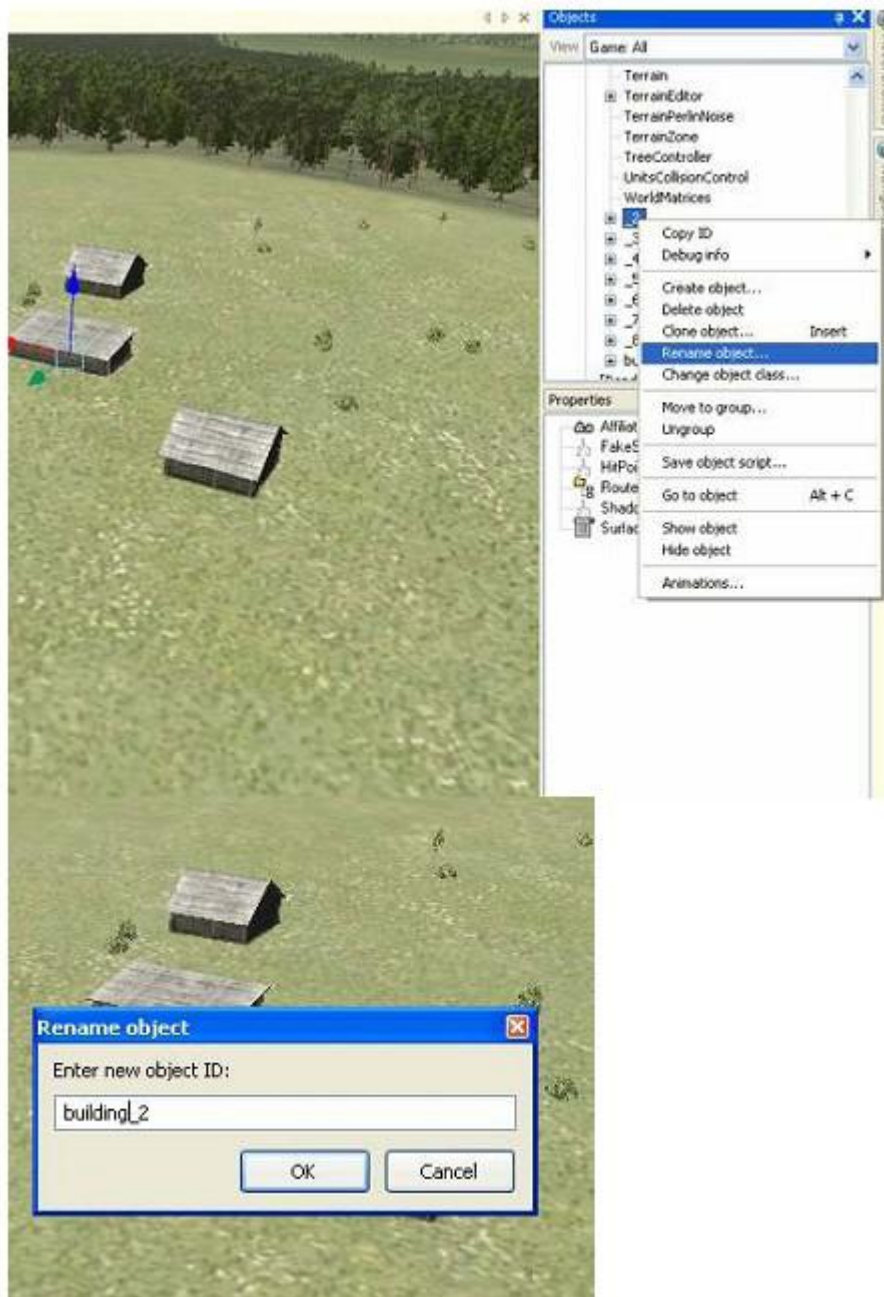


Similarly, create the rest of the house.

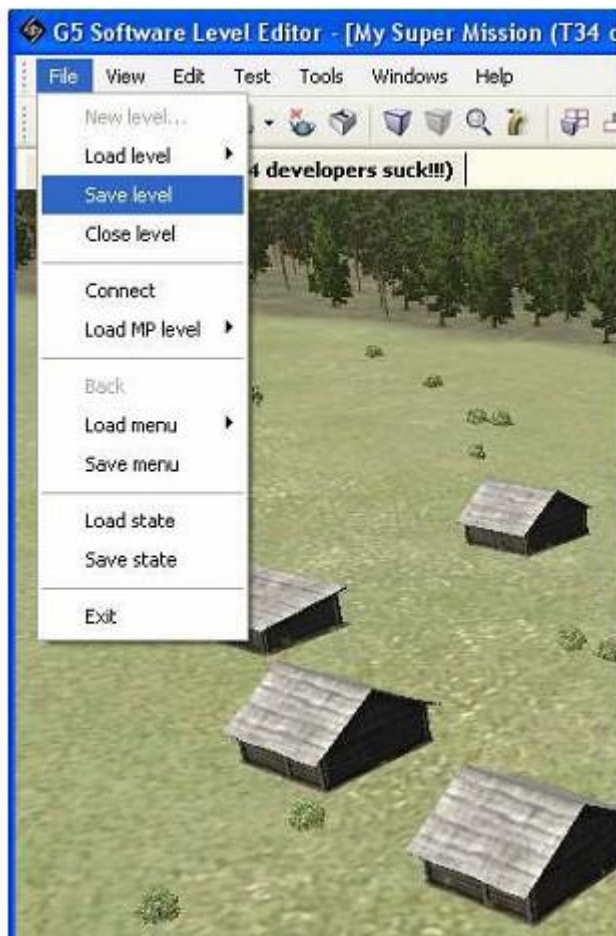


Names for objects created are automatically generated, but it is better to name them and not to leave the names the same as the auto naming.

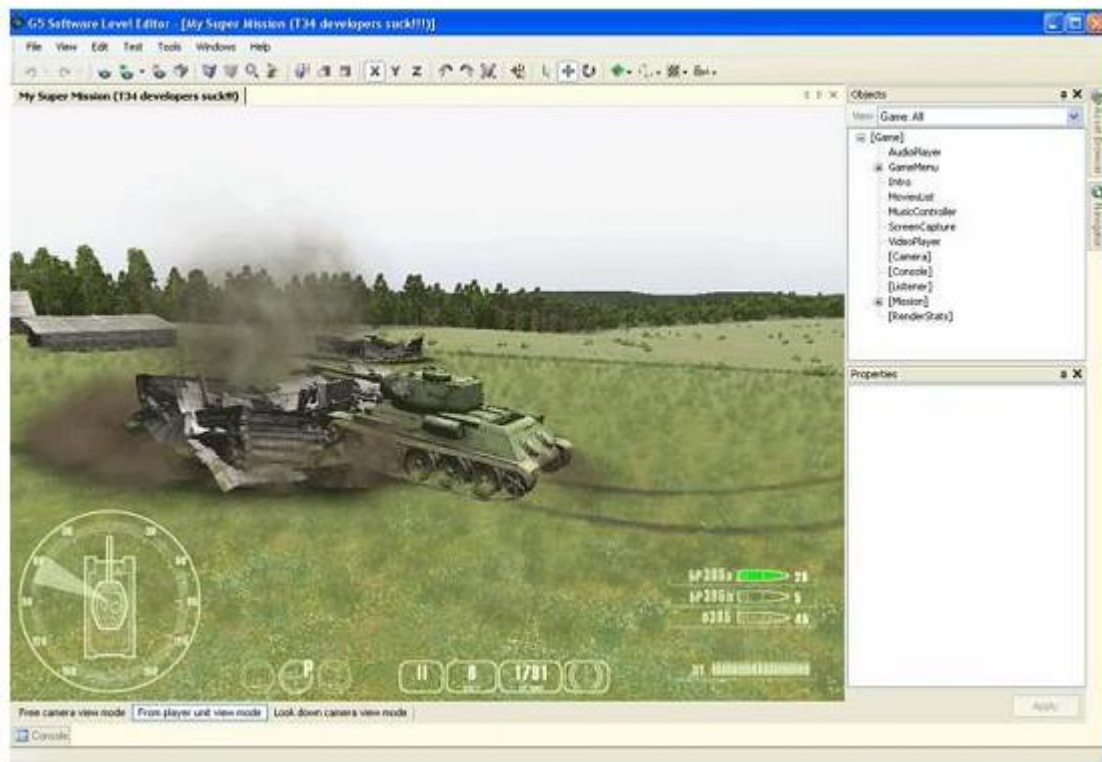
To rename them, right click mouse over the name of the object that we want to rename and select **"Rename object ..."**



Rename all objects that way. Once we have created a village, we will save our mission.

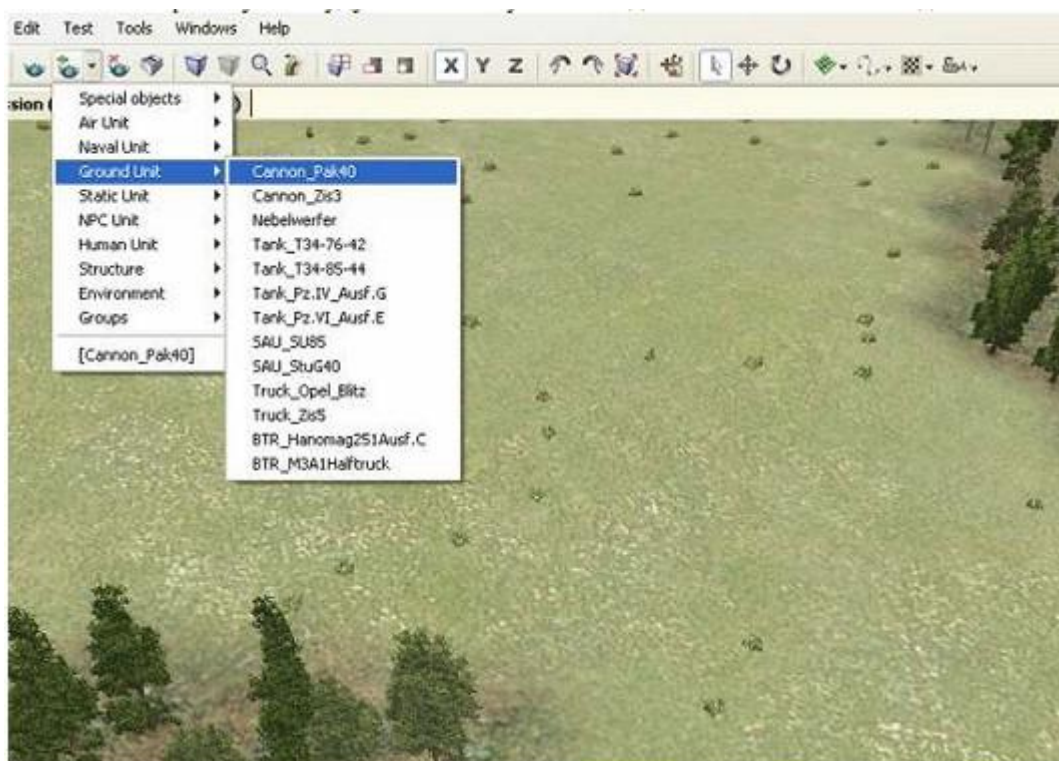


As I said earlier, all the objects that we add to the map in real time become interactive. We can drive up to our houses and destroy them.

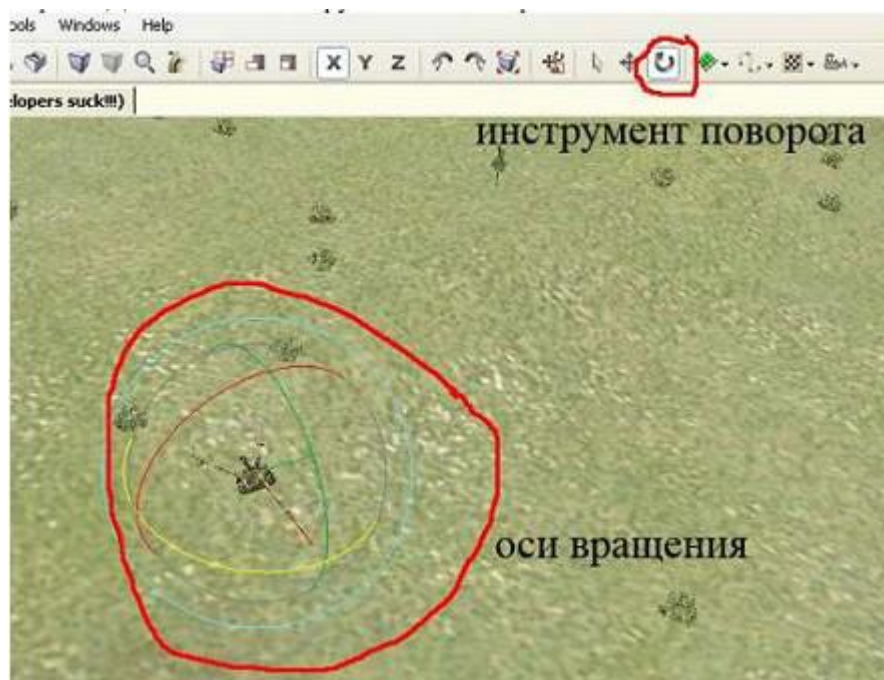


Step 8. Create a gun.

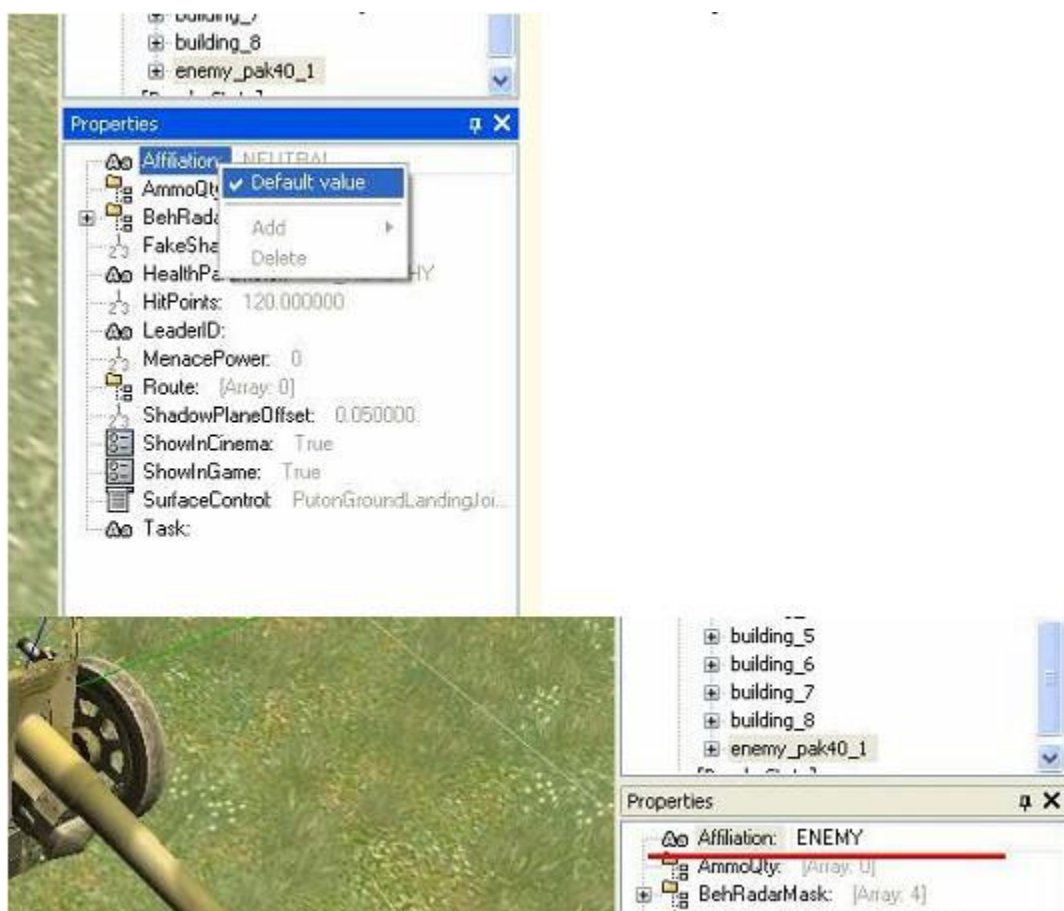
On the defensive line we will have 2 guns. They are created the same as houses.



Once created, you can turn it in the right direction, so that it looks at the player. This is done with the "turn" tool.



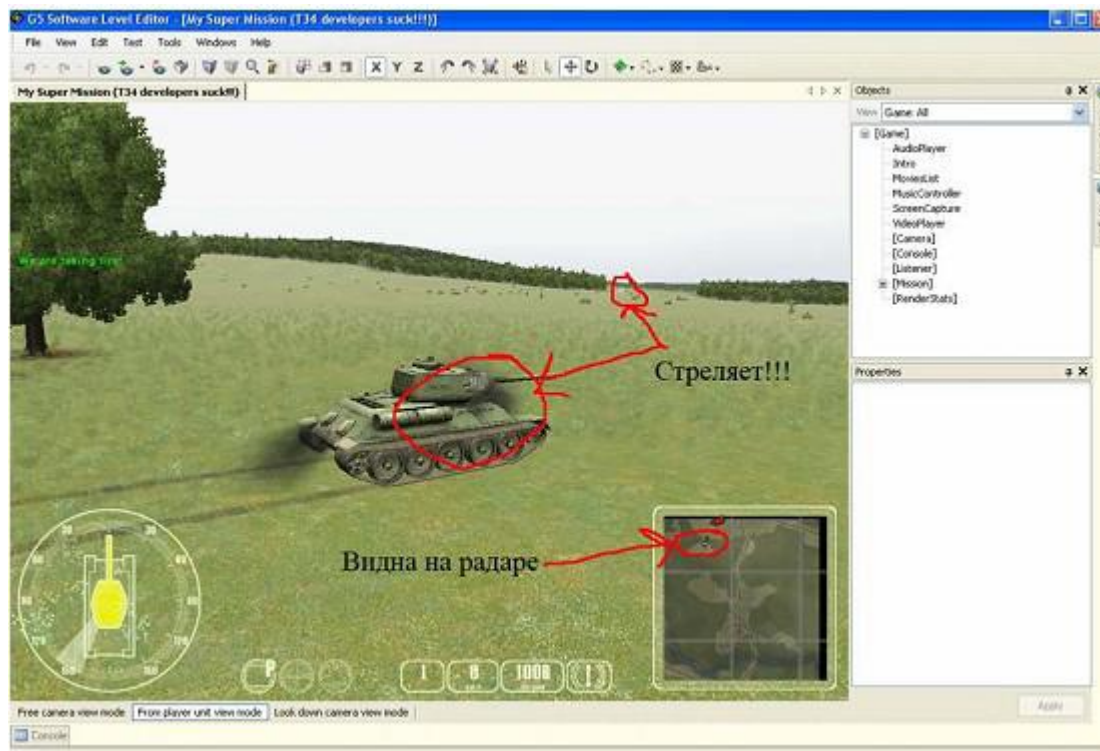
After this gun is still a static object. We need to tie it "brain" and set to "enemy" in its properties so that it fires at us, and not at their own troops. To do this we click the right button on the property and remove the check mark Affiliation with the Default value. After this you should prescribe the value of «Enemy» in this field.



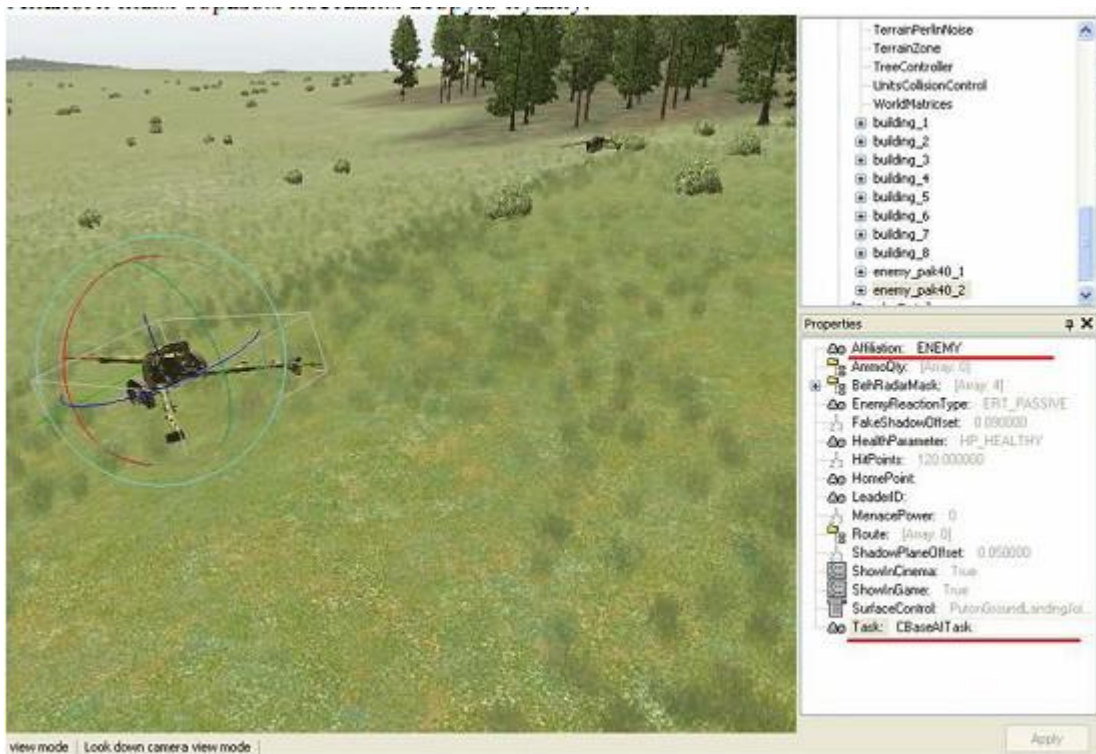
For the "brains" are responsible Tusky (Task), there is a small set of standard units for Tasco, that we can use. In this case, assign the gun «CBaseAITask».



Check how it works. save our level and load again. To activate the AI for the new unit, you must restart the mission. As soon as we drive out of the woods a gun will shoot at us, We have it so that it can also be destroyed.



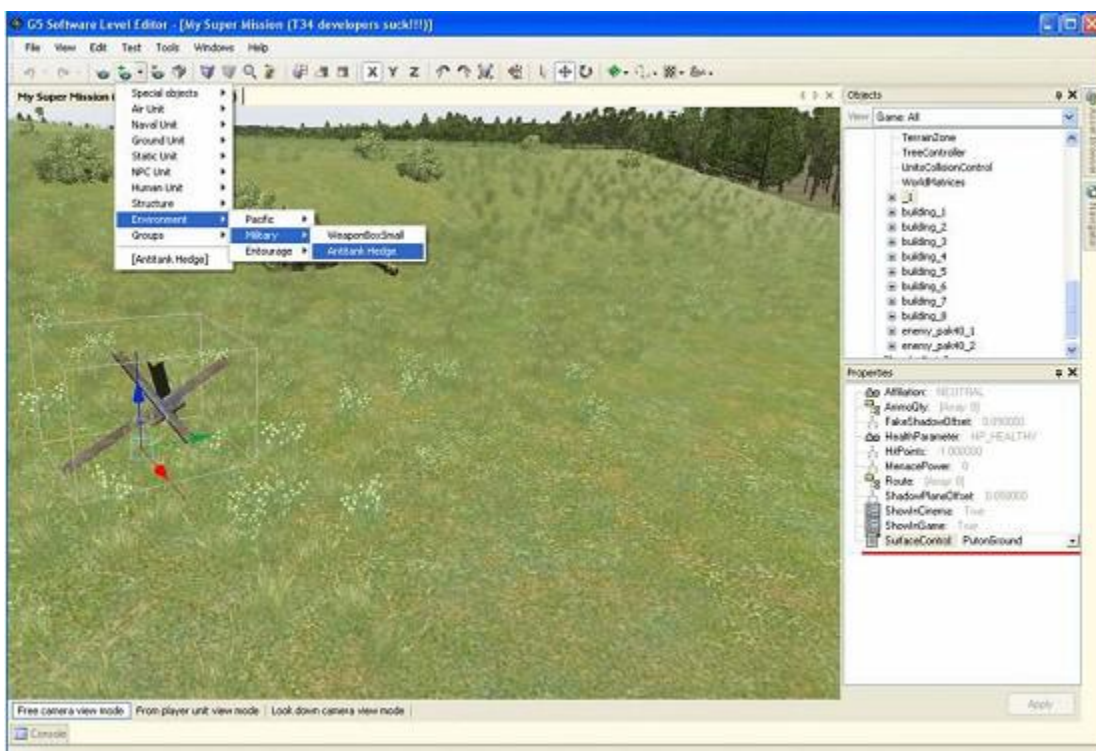
Similarly, we assign the second gun.



Step 9. Create the elements in the environment.

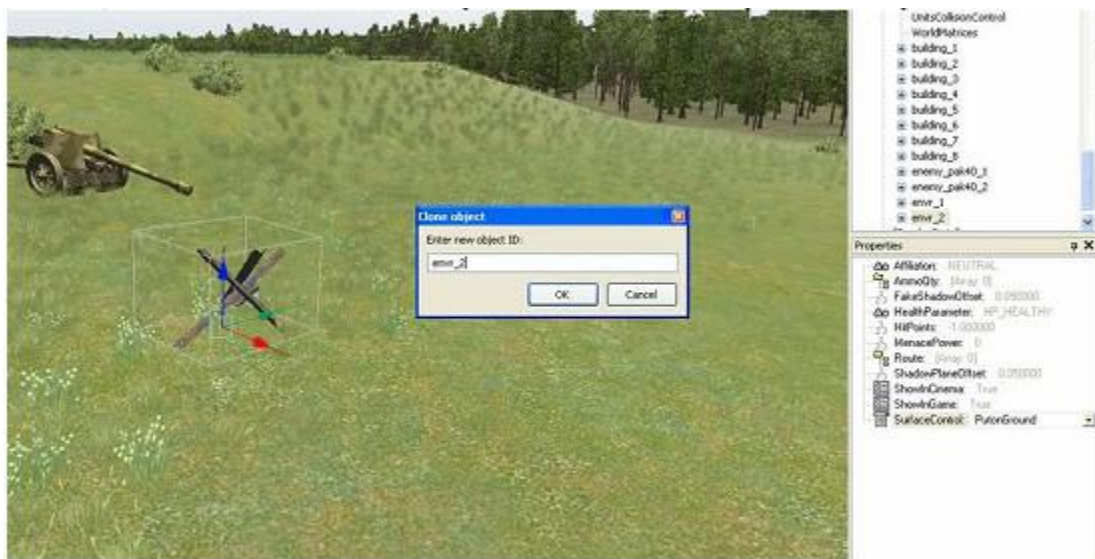
The game is good in parts! Therefore, we add elements to the environment, which diversify gaming space.

Placing anti-tank hedgehogs.

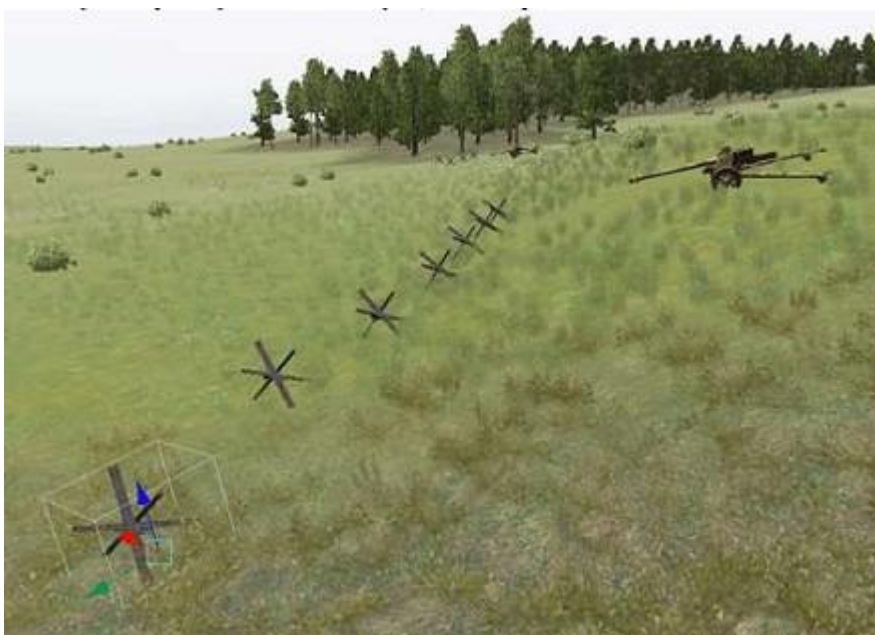


To make it convenient to move across the landscape, we can set the property "SurfaceControl" in value "PutonGround", then it will automatically positioned on the surface.

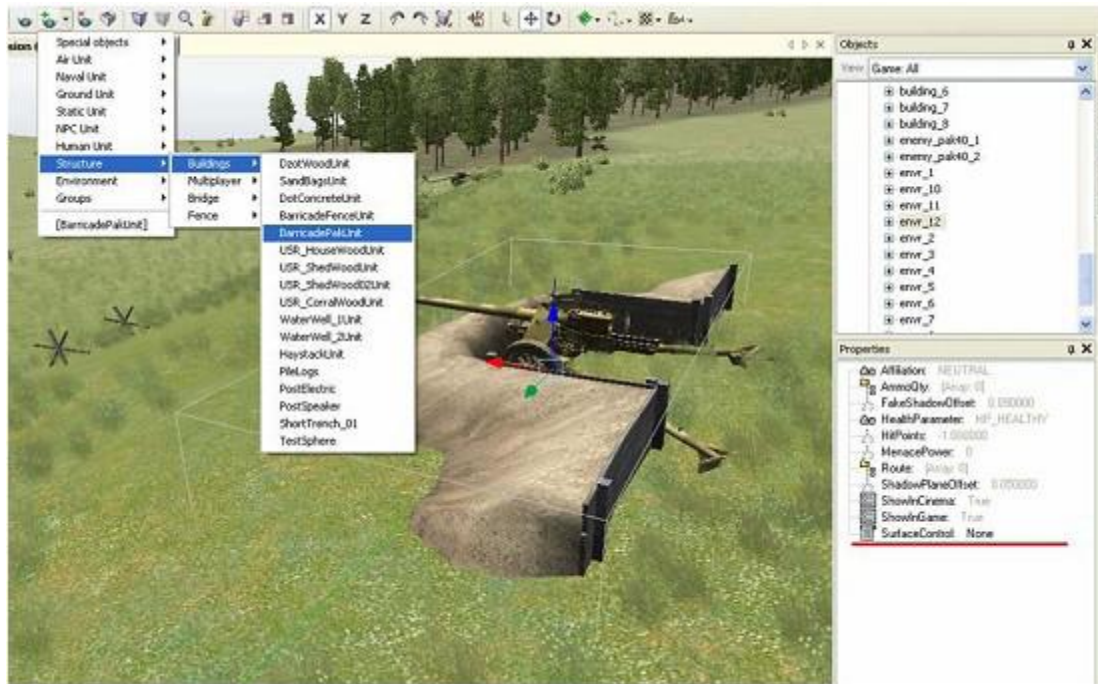
Because we want a lot of "Hedgehogs", I'll show you the way to clone objects. For this should either right click on the name of the object and choose «Clone object», or on the game screen, click Insert, then you will be prompted to enter a name for the new object and the object appears at the intersection of the camera view with the landscape.



By repeating this operation many times, we get the scene to look like this:

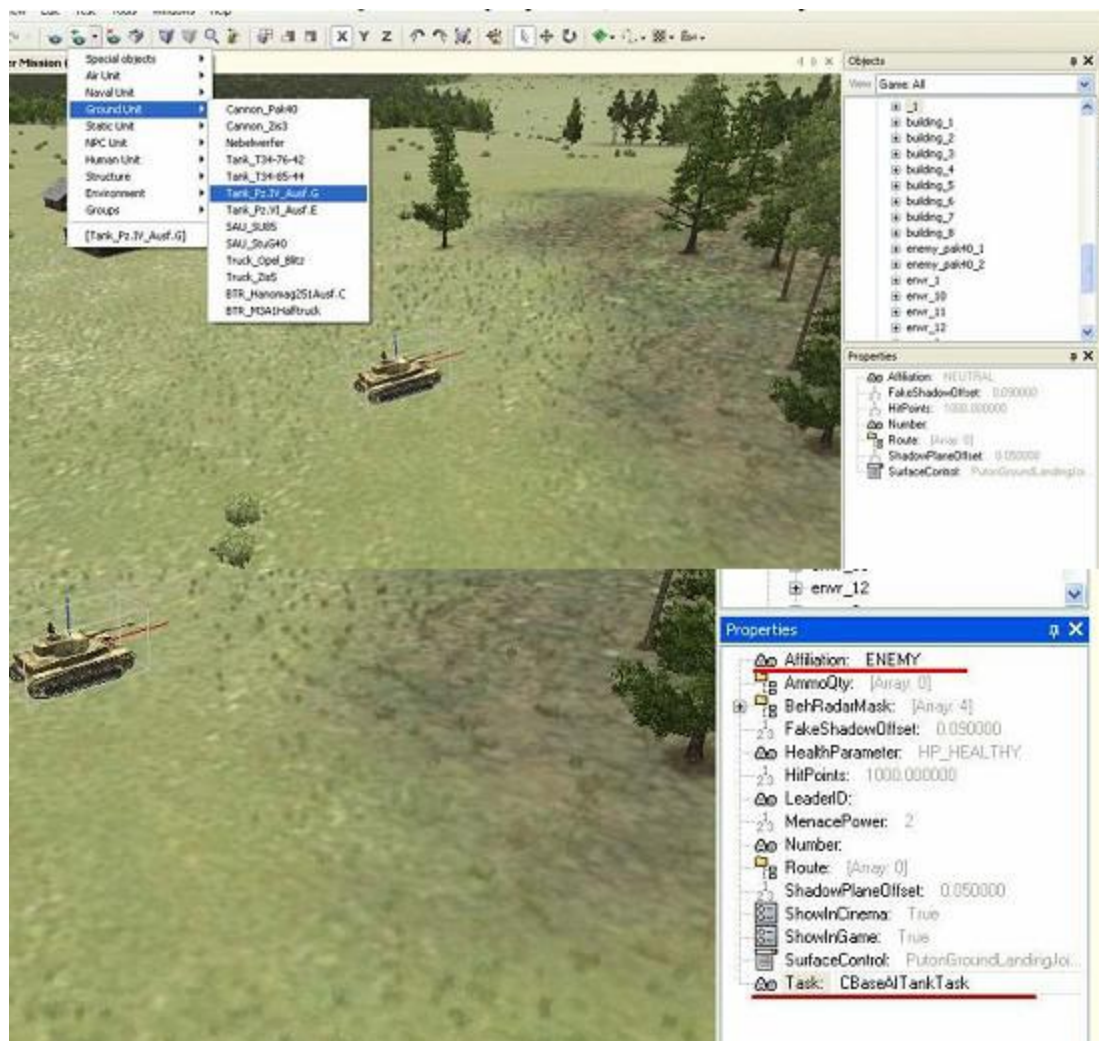


Next, close to the gun we will create a gun emplacement, and then assign the properties of "None"for "SurfaceControl". So it can sink in to the ground



Step 10. How to put an AI Tiger in.

When a player overcomes the defensive line, enemy tanks will be lying in wait to counter attack. For this we need to put it next to the village, and assign the necessary properties.

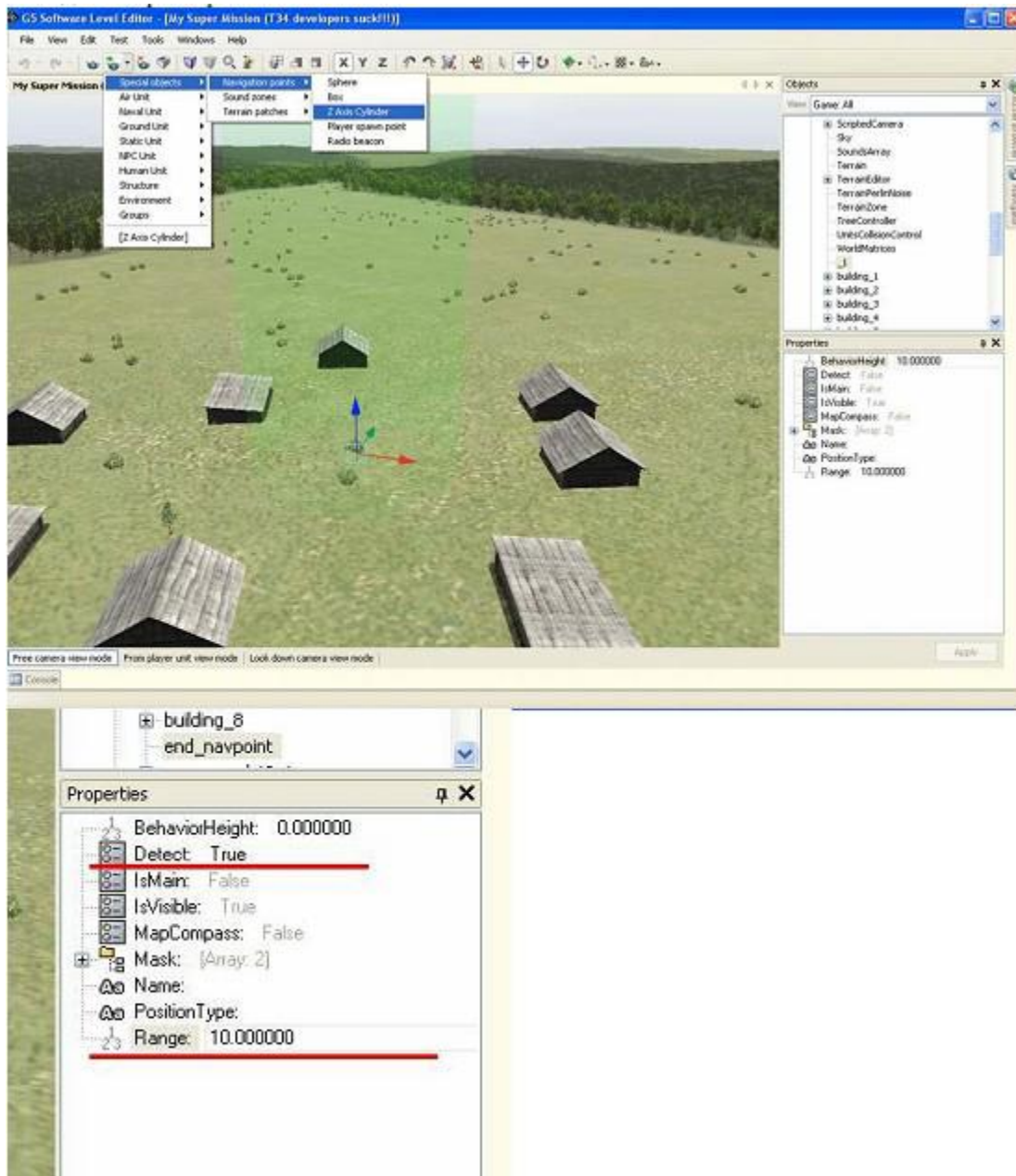


For the tanks we use CBaseAITankTask.

Do not forget to save the level as you add new objects. You can reload and check whether the tank is attacking you. Unfortunately, CBaseAITankTask does not include the movement from one place to another after the initialization is already must be set in scripts. But that's the next lesson.

Step 11. End of mission.

We want the mission to end when the player has entered the village. For this we you must install the trigger and handle the event raised when an object gets into the trigger.



To trigger fires, not just standing on the ground, it is necessary to activate Detect = True and set the radius of him. Handling all of the triggers are in the script of the mission. We need to check a unit came in and finish the mission, if it was a player.

Add the following lines to the script Mission.script.

```

void StartMission()
{
    // call inherited
    CSPMission::StartMission();

    Component console = new #GameController().GetObject(SOID_Console);
    // console.disablebeh();
    //console.showbehinf(true);
}

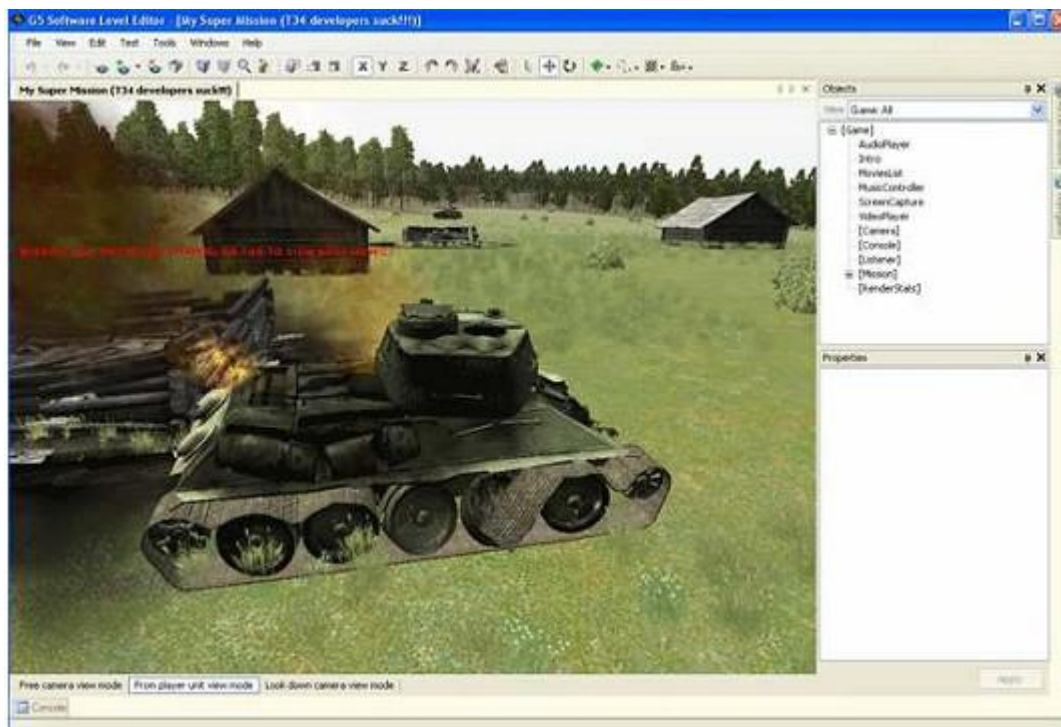
void OnObjectEnterNavPoint(String _NavPointID, String _ObjectID)
{
    if (_ObjectID == "MainPlayerUnit")
    {
        sendEvent(1.0, getIdentificator(this), "CloseMission", [HOSID_Completed]);
    }
}

void OnObjectLeaveNavPoint(String _NavPointID, String _ObjectID)
{
}

void Shutdown()
{
    CSPMission::Shutdown();
}
}

```

Now, if we drive into the village, the Triggers and the menu will be the end game. If we blow up, you will see an appropriate caption.



End!

Translated by Google translator and edited by PzU for the T34 vs Tiger community

Adding a new terrain and scenario from scratch

Step 1 File Set up

Copy DMX folder from MyTestMission on to the desk top and rename the following files changing DMX to DM15 or whatever number the death match scenarios' are up to, rename the DMX folder to DM15

DMX_Routerzone.bmp

DMX_micro.bmp

DMX_RouterZone.bmp

DMX_TerrainZone.bmp

DMX_Ind.tex

Forest_DMX.tex

Open the following Scritp files and replace DMX with DM15

Atmosphere

Content

Mission

MissionTest Strings

Terrain

WorlMatrices

Height maps have to conform to the following standards

1 Channel

16 Bit

IBM PC

0 Header

Interleaved unchecked

.RAW format

2049X2049 pixels

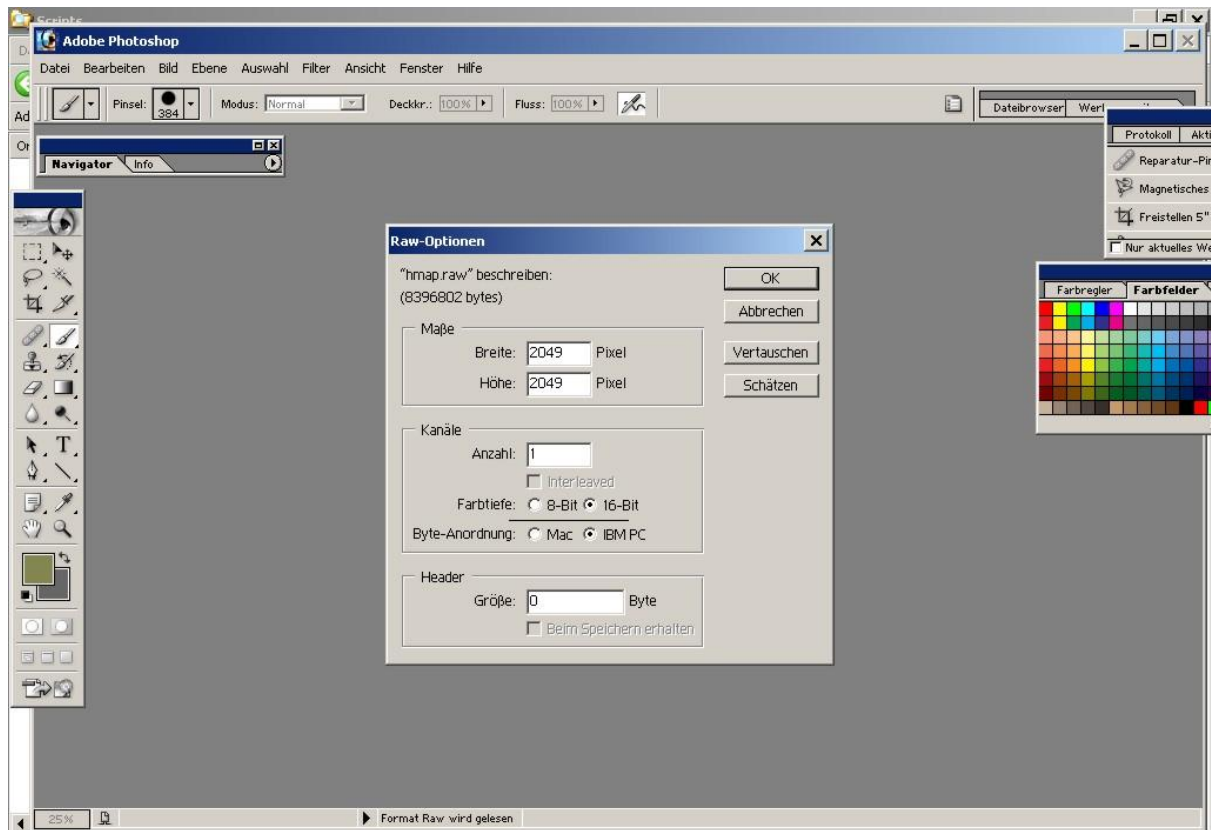
Greyscale

If you are converting a greyscale bitmap from 8 bit to 16bit RAW you will need to ensure the following actions are undertaken.

Open the bit map in Photoshop got to Image Mode select Greyscale, then select 16 bit, you will need to resize the image. This is done by using one of the Image tools, click on Image then click on Image Size, set the height and width to 2049x2049.

You will notice that the image has a spot effect of dark areas, this will represent individual peaks through out the map and make the map look like a piece of foam sound proofing or an egg box, it is vital this is removed as the editor will crash if the height map is too 'Noisy' i.e. too many little peaks all over the map. This can be undertaken using one of the filter tools, we have to reduce "Noise" on the map, using the noise tool, select "Reduce Noise", you will notice the dark spots will reduce on the map, you need to remove these completely so that each layer viewable layer does not have any dark black spots on it.

The map will now need to be saved as a RAW file, click on save as Photoshop Raw (*.Raw), change the file name to something other than the donor file. Click on save, select IBM PC and ensure the header is set to 0. (whilst the approach outlined above was undertaken on Photoshop this should work with other texture editing packages that support the RAW format)



There are two heightmaps one for the terrain and one for the water hmap.raw(terrain) and hwater.raw (water) the hmap.raw is the key file you can generate a heightmap in any height map tool but it must be saved in the following format 2049X2049 16bit and greyscale (only single channel) we have only managed to successfully convert save the raw files in Photoshop at present, the key script is the WorlMatrices script this contains two lines as follows

```
class CDM1WorldMatrices extends CWorldMatrices
{
    final static float MatrixWidth = 9000.0;
    final static float MatrixHeight = 9000.0;
}
```

this sets the height and width of the map in KMs the engine then stretches the height map over this matrix. The height map does not have to be square you can have odd shaped maps

Getting the new map in to the editor and selectable in the multiplayer screen

To get a custom terrain in the editor follow the standard procedure in the editor for creating a new map be it single or MP, as outlined above In order to get the map selectable from the editor menu you will need to edit the following script:

MenuConfig.Script

Scripts\Editor\MenuConfig.Script adding in the highlighted line.

```
//-----  
// List of MP missions in editor  
//-----  
final static Array MPMissionLoadList = [  
    ["DM 1 - Village Valley(4)"           , "CDM1Mission"],  
    ["DM 2 - Sunrise Field(8)"           , "CDM2Mission"],  
    ["DM 3 - Two Villages(10)"           , "CDM3Mission"],  
    ["DM 4 - Red Sunset(10)"             , "CDM4Mission"],  
    ["DM 5 - Road on Fire(16)"            , "CDM5Mission"],  
    ["DM 6 - Little Village(8)"           , "CDM6Mission"],  
    ["DM 7 - Fire Points(12)"             , "CDM7Mission"],  
    ["DM 8 - Command Post(12)"            , "CDM8Mission"],  
    ["DM 9 - Red Attacks(16)"             , "CDM9Mission"],  
    ["DM 10 - Defenders(16)"              , "CDM10Mission"],  
    ["DM 11 - Masters of the Hill(16)"     , "CDM11Mission"],  
    ["DM 12 - Critical Time(16)"           , "CDM12Mission"],  
    ["DM 13 - Scorced Earth(16)"           , "CDM13Mission"],  
    ["DM 14 - South Russia(16)"           , "CDM14Mission"],  
    ["DM 15 – North Lessay(16)"           , "CDM15Mission"],  
    ["CF 1 - Fire Points(8)"               , "CCF1Mission"],  
    ["CF 2 - Command Posts(12)"            , "CCF2Mission"],  
    ["CF 3 - Crtical Time(12)"             , "CCF3Mission"],  
    ["CF 4 - Battle for Bridgehead(16)"     , "CCF4Mission"],  
    ["CF 5 - Armour Hunters(16)"           , "CCF5Mission"],  
    ["CF 6 - Flags of the War(16)"         , "CCF6Mission"],  
    ["CF 7 - South Russia(16)"             , "CCF7Mission"]  
]
```

DMMission.Script

You will also have to edit the Scripts\Common\DMMission.script or else the editor will load the default MP map which is "FirePoitns".

```
final static Array MissionsID = [  
    ["CDM1Mission", 1],  
    ["CDM2Mission", 2],  
    ["CDM3Mission", 3],  
    ["CDM4Mission", 4],  
    ["CDM5Mission", 5],  
    ["CDM6Mission", 6],  
    ["CDM7Mission", 7],  
    ["CDM8Mission", 8],  
    ["CDM9Mission", 9],  
    ["CDM10Mission", 10],  
    ["CDM11Mission", 11],  
    ["CDM12Mission", 12],  
    ["CDM13Mission", 13],  
    ["CDM14Mission", 14],  
    ["CDM14Mission", 15]  
];
```

MenuSkin.Script

The Scripts\Menus\MenuSkin.Script will need to be updated This script defines the briefing map and its size Map can be created in XN view by saving it as .DDS then renaming the file extension to

```
new CUIOnDemandMaterial(  
    "DM14BriefingPic",  
    "Resources/DM14Map.tex",  
    0,  
    0,  
    512,  
    512  
)
```

```

new CUIOnDemandMaterial(
    "DM15BriefingPic",
    "Resources/DM15Map.tex",
    0,
    0,
    512,
    512
),

```

Game. Script

The game script has to have the new map added to the mp mapnames array, the file is in Scripts\Common\Game.Script

```

Array m_mapnames =
["CDM1Mission","CDM2Mission","CDM3Mission","CDM4Mission","CDM5Mission","CDM6Mission",
"CDM7Mission","CDM8Mission","CDM9Mission","CDM10Mission","CDM11Mission","CDM12Mission",
"CDM13Mission","CDM14Mission","CDM15Mission"];

```

MMP.Script

The game script has to have the new map added to the mp mapnames array, the file can be found in Resources\MPMissions.rsr.

```

Array m_mapnames =
["CDM1Mission","CDM2Mission","CDM3Mission","CDM4Mission","CDM5Mission","CDM6Mission",
"CDM7Mission","CDM8Mission","CDM9Mission","CDM10Mission","CDM11Mission","CDM12Mission",
"CDM13Mission","CDM14Mission","CDM15Mission"];

```

MultiPlayer Menu.Script

In order to make the map selectable from the multiplayer menu you will need to add the map in to the

```

static Array m_Maps = [

```

```

["CDM1Mission", 1],
["CDM2Mission", 2],
["CDM3Mission", 3],
["CDM4Mission", 4],
["CDM5Mission", 5],
["CDM6Mission", 6],
["CDM7Mission", 7],
["CDM8Mission", 8],
["CDM9Mission", 9],
["CDM10Mission", 10],
["CDM11Mission", 11],
["CDM12Mission", 12],
["CDM13Mission", 13],
["CDM14Mission", 14],
["CDM15Mission", 15]
];

```

MPMission.rsr

The multiplayer mission briefing screen text can be added to by editing the Resources\MPMission.rsr file, by adding an additional entry in either the deathmatch or team match parts of the file.

[DM14]

```

MissionName = "South Russia"

BriefingText = "Skirmish mission.\n 16 player maximum."

ObjectivesText = "Multiplayer DeathMatch Mode"

Objective01 = "Foggy"

```

[DM15]

```

MissionName = "North Lessay"

BriefingText = "Skirmish mission.\n 16 player maximum."

ObjectivesText = "Multiplayer DeathMatch Mode"

```

Objective01 = "Foggy"

Adding the in game map.

A map can be added so it can be displayed in the in game map window, this can be created by saving the maps as a .DDS and renaming the file extension to .TEX (this can be done using XN Viewer a free tool. The map is a 512X512 tile laid in the centre of a 1024x1024 black tile, all of the stock maps in TvT are 9kms square.

Texture/ ColorIndex/ MicroType

"Textures/Ind_micro01.tex", 0 Road, Mud, Ground
"Textures/Ind_micro02.tex", 1 Dry Ground Close Tex
"Textures/Ind_micro03.tex", 2 Medium Light Grass Close Tex
"Textures/Ind_micro04.tex", 3 Dark Grass and Ground Close Tex
"Textures/Ind_micro05.tex", 4 Light Grass Close Tex
"Textures/Ind_micro06.tex", 5 Field Close Tex
"Textures/Ind_micro07.tex", 6 Dark Grass Close Tex
"Textures/Ind_micro08.tex", 7 Forest Close Tex
"Textures/Ind_micro09.tex", 0 Road, Mud, Ground Near Tex
"Textures/Ind_micro10.tex", 1 Dry Ground Near Tex
"Textures/Ind_micro11.tex", 2 Medium Light Grass Near Tex
"Textures/Ind_micro12.tex", 3 Dark Grass and Ground Near Tex
"Textures/Ind_micro13.tex", 4 Light Grass Near Tex
"Textures/Ind_micro14.tex", 5 Field Near Tex
"Textures/Ind_micro15.tex", 6 Dark Grass Near Tex
"Textures/Ind_micro16.tex" 7 Forest Near Tex

"Textures/Village_micro01.tex", 0 Grass
"Textures/Village_micro02.tex", 1 Top-top grass
"Textures/Village_micro03.tex", 2 Downland grass
"Textures/Village_micro04.tex", 3 LittleRoad
"Textures/Village_micro05.tex", 4 Ground
"Textures/Village_micro06.tex", 5 Dry grass
"Textures/Village_micro07.tex", 6 Dirt

"Textures/Village_micro08.tex" 7 Graviy

"Textures/Village_micro01.tex", 0 Road Close Tex

"Textures/Village_micro02.tex", 1 Grass Close Tex

"Textures/Village_micro03.tex", 2 Forest Close Tex

"Textures/Village_micro04.tex", 3 Trample Close Tex

"Textures/Village_micro05.tex", 4 Mountain Close Tex

"Textures/Village_micro06.tex", 5 Sand Close Tex

"Textures/Village_micro07.tex", 6 Coast Close Tex

"Textures/Village_micro08.tex" 7 Bottom Close Tex

"Textures/Field_micro01.tex", 0 Road Close Tex

"Textures/Field_micro02.tex", 1 Grass Close Tex

"Textures/Field_micro03.tex", 2 Forest Close Tex

"Textures/Field_micro04.tex", 3 Trample Close Tex

"Textures/Field_micro05.tex", 4 Mountain Close Tex

"Textures/Field_micro06.tex", 5 Sand Close Tex

"Textures/Field_micro07.tex", 6 Coast Close Tex

"Textures/Field_micro08.tex" 7 Bottom Close Tex